

TECHNICAL REFERENCE MANUAL

TrustCore

Insurance comparison & offer platform

Architecture · Content engine · Product catalogue · Offer automation · API · Data · Security · Operations

Document	Version	Date	Basis	Classification
TC-TRM-001	1.0	3 November 2026	Code & data as of 22 Oct 2026	Confidential

Document control

PROPERTY	VALUE
Title	TrustCore — Technical Reference Manual
Identifier	TC-TRM-001
Version	1.0 — 3 November 2026
Status	Released
Classification	Confidential. Contains security findings; distribute to the technical owner and authorised contractors only.
Supersedes	TrustCore Project Overview, Developer Guide and User Guide (22 October 2026) for all technical content

Basis of this document

ARTEFACT	VERSION ANALYSED
Laravel back end	`Trustcore-Backend/` as deployed (without `vendor/`), Laravel 11.31, Sanctum 4
Angular front end	`Trustcore-Frontend/`, branch `trust-core-updates`, Angular 15.2, Nebular 11.0.1
Database	`TrustcoreDB.sql`, phpMyAdmin export of `ch379444_trustcore`, 22 Oct 2026 15:26 UTC, MySQL 8.0.42
Automation	`Poweflow-PAD_20261022162223.zip`, Power Automate package containing cloud flow *PAD*
Prior documentation	Project Overview, Developer Guide, User Guide (22 Oct 2026)

Audience and use

READER	RECOMMENDED PATH
Technical owner / CTO	Chapters 1, 2, 14 and 17
Back-end developer	Part II, chapters 8, 9, 11, 12 and appendices A–D
Front-end developer	Chapters 5.2, 10 and 11
Automation engineer	Chapter 7 and the integration endpoints in chapter 9
Operations	Chapters 3, 16 and 17

Conventions

Identifiers from code and database are set in `monospace`. “At export” refers to the database snapshot above. Figures marked as inferred describe behaviour outside the delivered code that is implied by the interfaces it exposes. Line references point to the delivered source files. Secret values (trigger signatures, tokens, credentials) are deliberately not reproduced.

Contents

PART ONE · SYSTEM OVERVIEW

1 Executive summary 7
1.1 Platform at a glance 7
1.2 What the platform does 7
1.3 Architecture in one paragraph 7
1.4 Key engineering characteristics 8
1.5 Headline findings 8
1.6 Sources and method 8

2 System context 9
2.1 Actors 9
2.2 External systems 9
2.3 Trust boundaries 10

3 Containers and deployment 11
3.1 Containers 11
3.2 Physical deployment 12
3.3 Environments 12

4 Technology stack 14
4.1 Front-end dependencies 14
4.2 Back-end dependencies 14
4.3 Code volume 15

PART TWO · THE CORE ENGINE

5 Collections engine 17
5.1 Meta-model 17
5.2 The column grammar 18
5.3 Field management 19
5.4 Relation resolution 20
5.5 Entry operations 21

6 Templates and product catalogue 22
6.1 Template anatomy 22
6.2 Product catalogue 23
6.3 Creating a template 24
6.4 Lookup collections 25

7 Offer pipeline and automation 26
7.1 End-to-end sequence 26
7.2 Offer states 27
7.3 Power Automate cloud flow 28
7.4 SMS one-time-code relay 29
7.5 Failure modes 29

PART THREE · APPLICATION LAYERS

8 Back-end architecture 31

8.1 Request life cycle 31
8.2 Component structure 32
8.3 Error handling conventions 33

9 API reference 34
9.1 Conventions 34
9.2 Authentication 34
9.3 Collections and schema 34
9.4 Entries 35
9.5 Users, roles, settings, reports 35
9.6 Integration endpoints 35
9.7 Session routes and framework endpoints 35
9.8 Example exchange 36

10 Front-end architecture 37
10.1 Module and component structure 37
10.2 Routing 37
10.3 The generic screens 38
10.4 Client-side state 38
10.5 Internationalisation 39
10.6 Build and toolchain 39

11 Identity and access control 40
11.1 Sign-in 40
11.2 Role model 40
11.3 Enforcement layers 41
11.4 Data scoping 42

PART FOUR · DATA

12 Database architecture 44
12.1 Schema layers 44
12.2 Core entity relationships 45
12.3 Volumes 46
12.4 Integrity assessment 46
12.5 Data protection 47

13 Reporting 48
13.1 Endpoint 48
13.2 Response shape 48
13.3 Labels and translation 48

PART FIVE · SECURITY, QUALITY AND OPERATIONS

14 Security assessment 50
14.1 Method 50
14.2 Risk heat map 50
14.3 Summary 50
14.4 Findings 51
14.5 Secrets and exposed artefacts 54

15 Code quality and technical debt 55
15.1 Strengths 55
15.2 Debt register 55

15.3 Dependency currency 55

16 Operations runbook 57
16.1 Local workstation 57
16.2 Release procedure 58
16.3 Configuration reference 58
16.4 Routine operations 58
16.5 Troubleshooting 59

17 Remediation roadmap 60
17.1 Phase 0 — Contain (weeks 1–2) 60
17.2 Phase 1 — Authorise (weeks 3–6) 60
17.3 Phase 2 — Operate (weeks 7–9) 61
17.4 Phase 3 — Modernise (weeks 10–16) 61
17.5 Definition of done 61

APPENDICES · REFERENCE DATA

A Table catalogue 63

B Template field dictionary 66

C Lookup collections 70

D Collection registry 72

E Source inventory 75

F Glossary 78

PART ONE

System Overview

What TrustCore is, the boundaries of the platform, the systems it depends on and the technology it is built from. Read this part first to orient yourself before working on any single layer.

1	Executive summary
2	System context
3	Containers and deployment
4	Technology stack

1 Executive summary

TrustCore is a Swiss insurance comparison and offer platform. Advisors capture customer profiles, configure multi-line insurance offers against insurer-specific templates and hand the offer to an automation pipeline that produces a signed-off PDF for the customer. The platform is built around a schema-on-demand content engine that lets staff extend the product catalogue without code changes.

1.1 Platform at a glance

97
Database tables
14 platform, 3 meta-model, 64 lookup, 16 template instances

2,206
Typed columns
1,135 relational (`rel_`) columns resolved at runtime

82
Collections
registered content types, each backed by its own physical table

19
Product lines
basic KVG plus 18 supplemental VVG lines in one offer template

34
REST endpoints
under `/api`; 7 more session routes under `/`

129
Fields per template
largest insurer template (Groupe Mutuel), one configuration row

4
UI languages
EN · FR · DE · IT across 281 translated labels

20
Registered risks
4 critical, 7 high — see chapter 14

1.2 What the platform does

An **advisor** (role *employee*) maintains a portfolio of assigned customers. For each customer the advisor creates an **offer**: a title, the customer, and an **insurer template** — for example *Groupe Mutuel* or *Nur KVG*. The template pre-configures up to 129 fields that describe which product lines are included (hospitalisation, dental, daily allowance, legal protection, CyberProtect and so on), the chosen insurance model, coverage level, deductible, waiting period and the date each line comes into force.

On submission the Laravel API stores the offer in state *Creating Offer* and triggers a Microsoft Power Automate cloud flow with the offer ID. The flow launches an unattended Power Automate Desktop robot that operates the insurer's own web portal, completes a one-time-code sign-in relayed through TrustCore's SMS endpoint, and posts the resulting PDF back to TrustCore. The offer then switches to *Ready*; failures and timeouts are reported back as statuses.

Customers can sign in and see offers; **administrators** control users, roles, the content model, branding and every catalogue entry.

1.3 Architecture in one paragraph

A single-page Angular 15 application (built on the ngx-admin / Nebular template) talks JSON over HTTPS to a Laravel 11 API secured with Sanctum bearer tokens. Both are deployed side by side on Hoststar shared hosting under `trustcore.ch`, with Apache rewrite rules splitting SPA routes from API calls. MySQL 8 stores a small fixed schema plus dozens of tables that the application itself creates, alters and drops at runtime. Offer document production runs outside the code base in Microsoft Power Automate (cloud flow + desktop flow).

1.4 Key engineering characteristics

CHARACTERISTIC	DESCRIPTION
Schema-on-demand	New content types and fields are created through the UI and immediately become real SQL tables and typed columns (`CREATE TABLE` , `ALTER TABLE`) — no migrations, no deployment.
Self-describing columns	A column name encodes its type, its required flag and, for relations, its target table (`rel_gender_req_col_gender`). Backend validation, relation resolution and frontend widget selection all parse the same grammar.
One generic UI	A single table component and a single form component render every collection by handle. There is no dedicated offers screen — offers, customers, insurers and products are all the same code path.
Asynchronous document pipeline	PDF production is delegated to an RPA robot through a webhook round-trip with two callback endpoints and an SMS-based OTP relay.
Role-scoped data	Advisors see customers assigned to them, offers of those customers, and the templates and insurer credentials they created.

1.5 Headline findings

BEFORE ANY FURTHER PRODUCTION USE

The review identified four critical issues: an unauthenticated file-upload endpoint that writes into the public web folder with the uploader's file extension; user accounts that any signed-in user can modify (including role assignment); authorisation decisions based on a role sent by the browser in the query string; and raw SQL concatenation in template creation. Chapter 14 documents each finding with evidence and remediation, and chapter 17 sequences the fixes into a 16-week plan that closes every critical item within the first six weeks.

1.6 Sources and method

This reference was produced from a complete reading of the handed-over artefacts: the Laravel back end (27 PHP application files, 4 route files, 14 migrations, configuration), the Angular front end (`trust-core-updates` branch), the production database export of 22 October 2026 (97 tables, parsed table by table), and the exported Power Automate package `PAD_20261022162223` . Figures quoted as “at export” refer to that database snapshot. No personal data from the export is reproduced in this document.

2 System context

TrustCore sits between three human audiences and five external systems. Only the central platform is contained in the code base; the automation chain and every insurer-facing interaction live in Microsoft's cloud and on a Windows robot host.

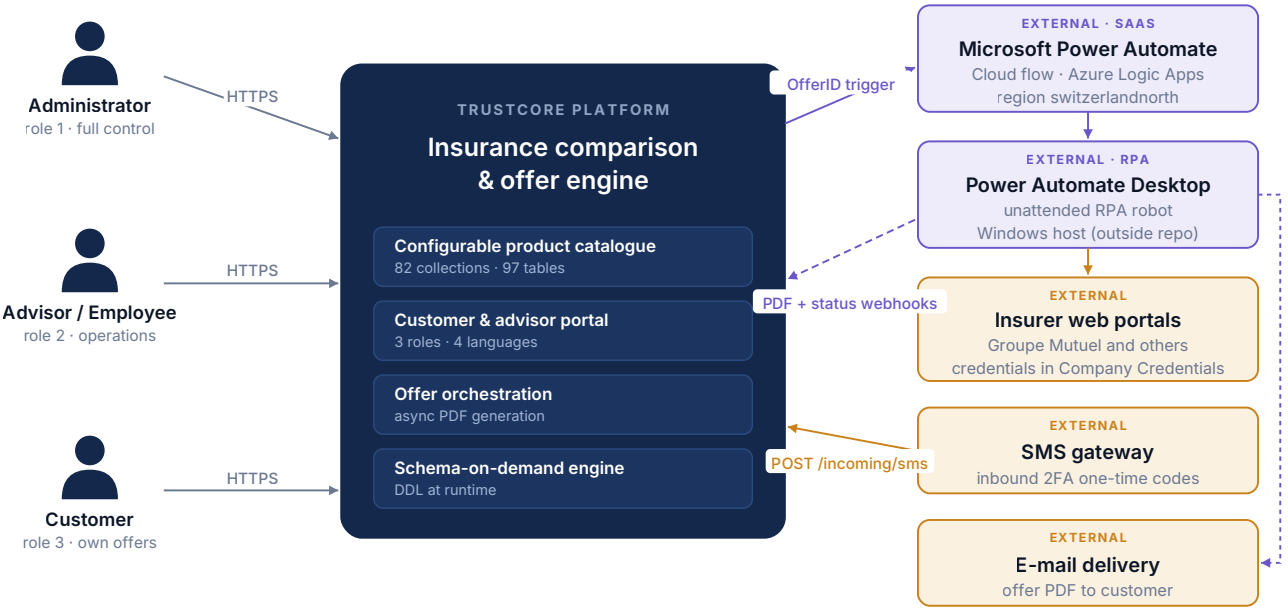


Figure 1 System context (C4 level 1). Actors on the left, the TrustCore platform in the centre, and the external systems that complete the offer life cycle on the right.

2.1 Actors

ACTOR	ROLE ID	RESPONSIBILITIES	AT EXPORT
Administrator	`1 · admin`	Users and role assignment, content model (collections, fields), site branding, all catalogue data, all offers.	6 accounts
Employee / advisor	`2 · employee`	Own customers, offers for those customers, own templates, own insurer portal credentials, dashboard report for own portfolio.	3 accounts
Customer	`3 · customer`	Offers and personal settings (language, theme). Default role when none is supplied at registration.	11 accounts
Unassigned	–	Accounts present in `users` without a row in `user_roles`. The front end assumes `roles[0]` exists and fails for these accounts.	3 accounts

2.2 External systems

SYSTEM	INTERFACE	PURPOSE AND CONTRACT
Power Automate cloud flow	HTTPS trigger · Azure Logic Apps, region *switzerlandnorth*	Receives `{"OfferID": int}` from Laravel; launches the desktop flow; reports *Failed* or *Timeout* back. Trigger authentication type *All* — the signed URL is the only credential.

SYSTEM	INTERFACE	PURPOSE AND CONTRACT
Power Automate Desktop	Connector `shared_uiflow` · `RunUIFlow_V2`	Unattended robot (desktop flow `ca288a0d-…`) that drives the insurer portal, uploads the PDF through the offer webhook and e-mails the customer. Its script is not part of the handed-over package.
Insurer web portals	Browser automation	Offer calculation and PDF issue. Portal logins are stored per advisor in the *Company Credentials* collection (`insurance_companies_meta`).
SMS gateway	`POST /api/incoming/sms`	Delivers inbound SMS as JSON; TrustCore extracts the first six-digit code and exposes it on `GET /api/latest-otp` for the robot.
E-mail delivery	Via the robot / flow	Sends the PDF to the customer. Laravel's mailer is configured as `log` and is used only for password-reset links.

2.3 Trust boundaries

Three boundaries matter for every design decision in this platform:

1. **Browser ↔ API.** Everything the browser sends — including its claimed role and user id — must be treated as untrusted. Today the API trusts both (chapter 11).
2. **API ↔ automation.** Calls in both directions cross the public internet. The outbound trigger is protected only by the URL signature; inbound callbacks are not authenticated at all (chapter 7).
3. **API ↔ database schema.** Because the application issues DDL, any user who can reach the schema endpoints can change the structure of the production database (chapter 5).

3 Containers and deployment

Two deployable units — a compiled Angular bundle and a Laravel application — share one Apache virtual host on shared hosting. A pair of .htaccess files decides which unit answers each request.

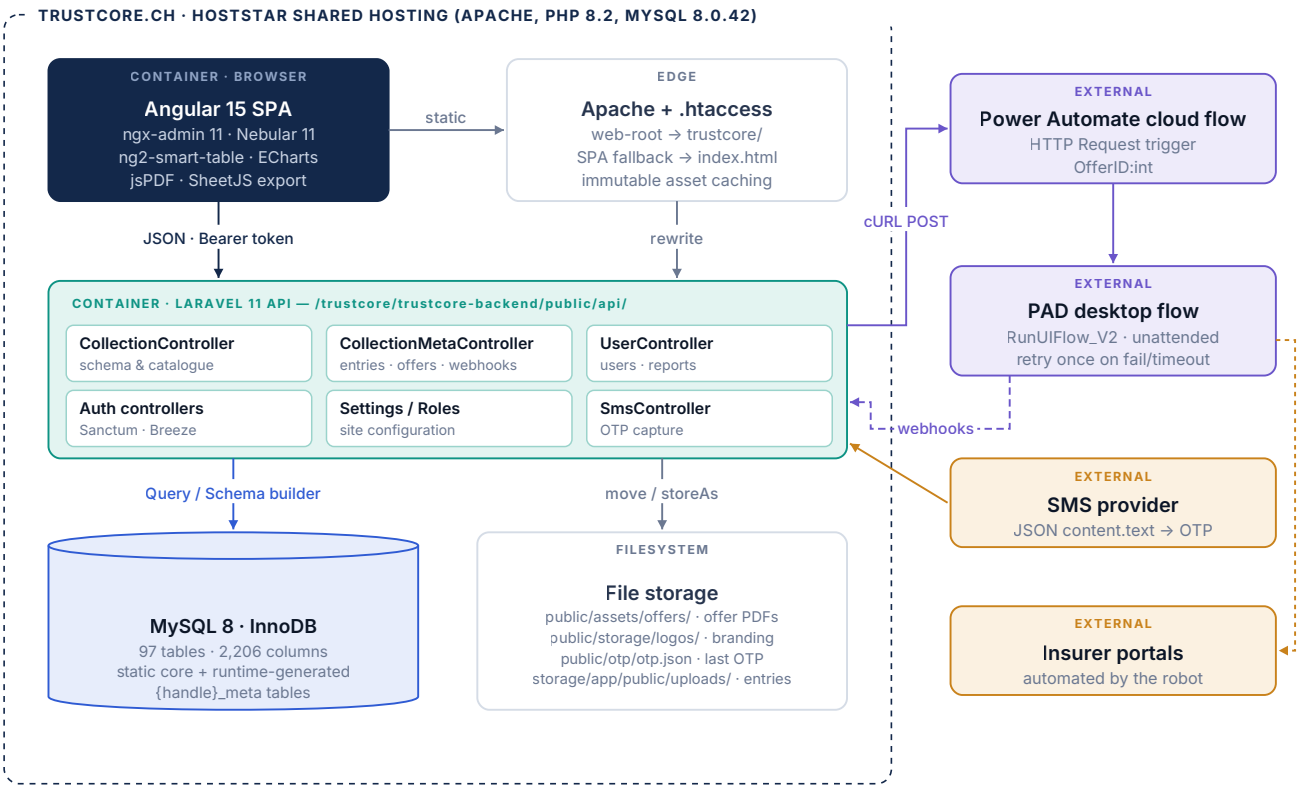


Figure 2 Container view (C4 level 2). The Laravel container is shown with its six controller groups; file storage is split across four public and private locations.

3.1 Containers

CONTAINER	TECHNOLOGY	RESPONSIBILITY
Single-page application	Angular 15.2, Nebular 11, ngx-admin 11	All user interaction: authentication screens, menus, generic tables and forms, field builder, user management, settings, dashboard report, PDF / Excel export.
REST API	Laravel 11.31 on PHP ≥ 8.2, Sanctum 4	Authentication, content engine (DDL and DML), relation resolution, user management, reports, offer trigger and webhooks, SMS capture.
Relational database	MySQL 8.0.42, InnoDB, utf8mb4	Static platform tables plus all runtime-generated `{handle}_meta` tables. Laravel cache and queue tables also live here.
File storage	Local disk under `public/` and `storage/`	Offer PDFs (113 files at export for 52 offers — older versions are never removed), company logo, entry uploads, latest OTP.

3.2 Physical deployment

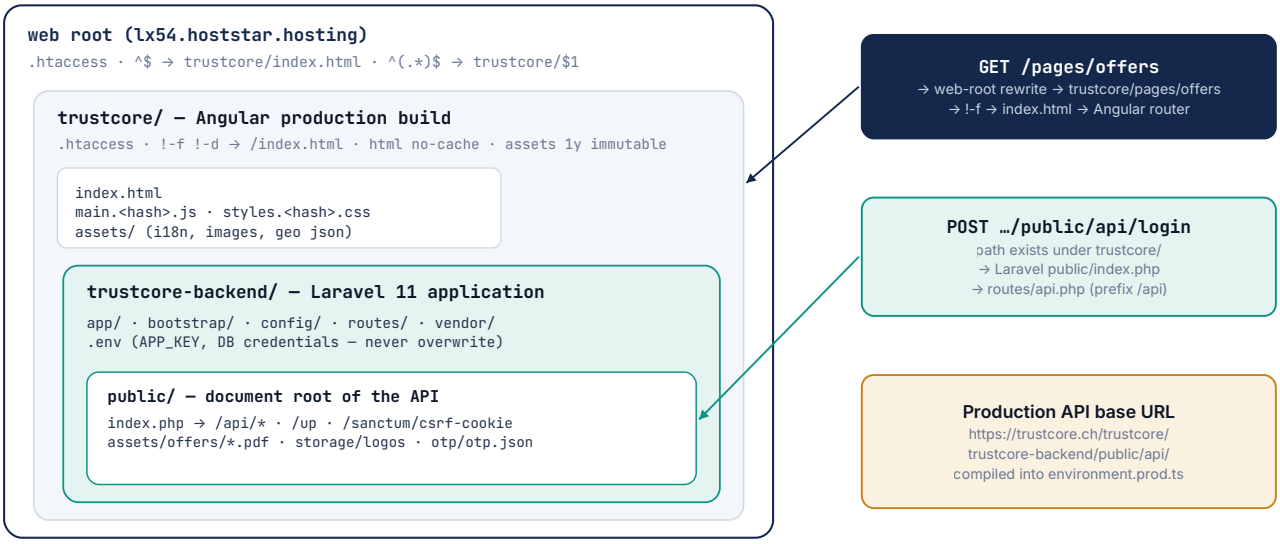


Figure 3 Directory topology on the Hoststar host and how two representative requests are routed.

The web root contains a single `.htaccess` that rewrites the empty path to `trustcore/index.html` and every other path into `trustcore/`. Inside that folder a second `.htaccess` serves existing files directly and falls back to `index.html` so that Angular's router handles deep links. HTML is sent with `no-cache`; hashed JavaScript, CSS, fonts and images are cached for one year as `immutable`.

The Laravel application lives inside the Angular folder at `trustcore/trustcore-backend/`. Because Apache serves existing paths, any request to `…/trustcore-backend/public/…` reaches Laravel's front controller, which gives the API its production base URL `https://trustcore.ch/trustcore/trustcore-backend/public/api/`. This address is compiled into `environment.prod.ts`; moving the back end requires a front-end rebuild.

Hosting facts

ITEM	VALUE
Host	Hoststar shared hosting, server `lx54.hoststar.hosting`, control panel my.hoststar.ch
Database server	MySQL 8.0.42 (Ubuntu 20.04 build), schema `ch379444_trustcore`, administered through phpMyAdmin 5.2.1
PHP	8.2.28 at export time
Access	SFTP (transfer only). Shell access is granted on request as a short-lived password.
Laravel runtime settings	file sessions (120 min), database cache and queue, log mailer, `APP_DEBUG=true` in production
Health check	`GET /up` (Laravel 11 built-in)

3.3 Environments

ENVIRONMENT	FRONT-END TARGET	NOTES
Local	`http://127.0.0.1:8000/api/`	Angular dev server on port 4200; `php artisan serve` on 8000. Requires Node 18 (ngcc fails on newer Node) and `npm install --legacy-peer-deps`.
Production	`https://trustcore.ch/trustcore/trustcore-backend/public/api/`	Only a compiled bundle was found on the server; the source of record is the `trust-core-updates` branch.

ENVIRONMENT	FRONT-END TARGET	NOTES
Legacy	`trustcode.techsolsint.com`	Earlier host. Still referenced by the exported Power Automate flow's failure callbacks (chapter 7) and by the old `master` branch.

NO STAGING ENVIRONMENT

Because the flow trigger URL is hard-coded, any non-production copy that creates an offer will start the production robot with an offer id that, in production, belongs to a different customer. Until the URL is moved to configuration, offers must only be created in production.

4 Technology stack

The stack is conventional at each layer — Angular, Laravel, MySQL — and unconventional in how the layers cooperate: the database structure is data, and the UI is generated from it.

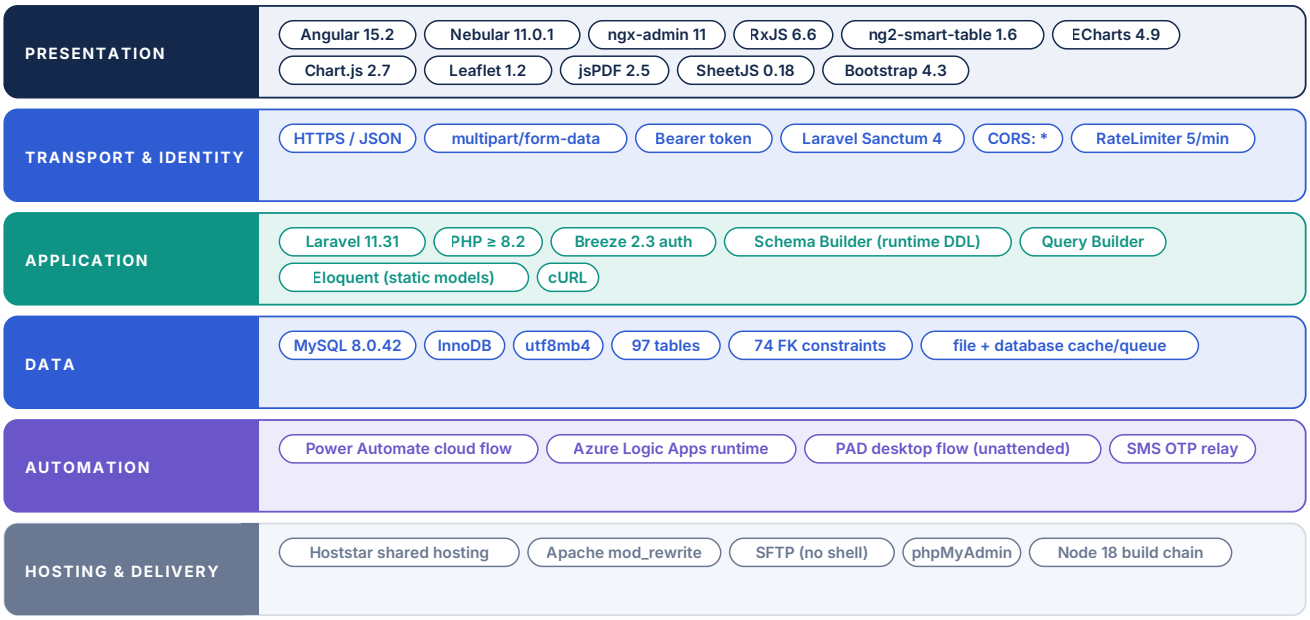


Figure 4 Technology stack by layer, with the versions declared in package.json, composer.json and observed on the production host.

4.1 Front-end dependencies

PACKAGE	VERSION	USE IN TRUSTCORE
`@angular/*`	15.2.x	Framework, router, reactive forms, HTTP client
`@nebular/theme · auth · security`	11.0.1	Layout, menu, four themes (default, dark, cosmic, corporate), authentication screens and strategy base class
`ng2-smart-table`	1.6.0	Every data grid: collections, entries, users, roles
`echarts / ngx-echarts`	4.9 / 4.2	Dashboard 12-month report
`jspdf · jspdf-autotable`	2.5 / 3.8	PDF export of user and customer tables
`xlsx · file-saver`	0.18 / 2.0	Excel export
`@angular/google-maps`	12.2.13	Demo map only; three majors behind Angular, forces `--legacy-peer-deps`
`leaflet · chart.js · ckeditor · tinymce`	1.2 · 2.7 · 4.7 · 4.5	Inherited from the ngx-admin demo pages; not used by TrustCore screens
`rxjs`	6.6.2 (+ rxjs-compat)	Service observables

4.2 Back-end dependencies

PACKAGE	VERSION	USE IN TRUSTCORE
`laravel/framework`	^11.31	Routing, Query Builder, Schema Builder (runtime DDL), validation, filesystem
`laravel/sanctum`	^4.0	Personal access tokens (`HasApiTokens` on `User`); token expiration `null`

PACKAGE	VERSION	USE IN TRUSTCORE
`laravel/breeze` (dev)	^2.3	API-only auth scaffolding: login request with rate limiting, password reset, e-mail verification
`laravel/tinker`	^2.9	REPL
`phpunit/phpunit` (dev)	^11.0.1	Only the Breeze sample tests exist; none covers TrustCore logic

4.3 Code volume

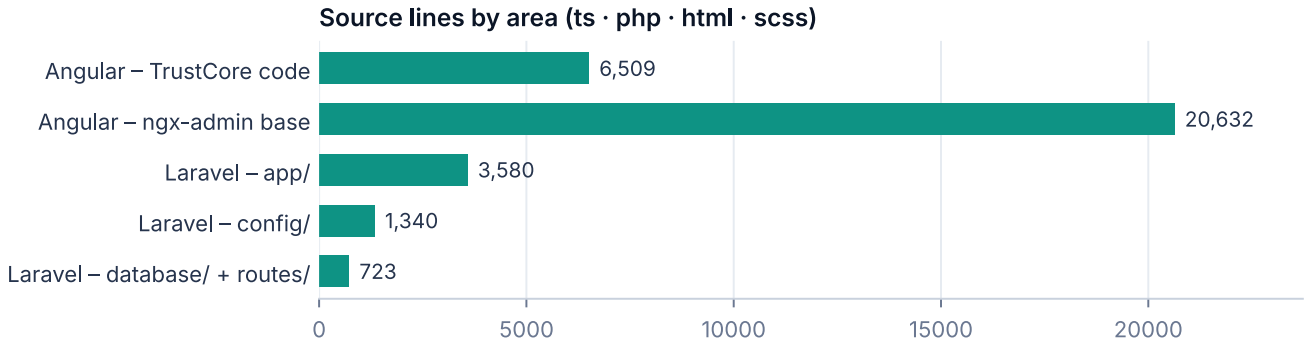


Figure 5 Source lines per area. The ngx-admin template contributes most of the Angular line count; TrustCore's own front-end code is about 6,500 lines across components, services, guard, strategy and the users module.

PART TWO

II

The Core Engine

The three mechanisms that make TrustCore different from a conventional CRUD application: the runtime-generated content model, the insurer template system that encodes the Swiss product catalogue, and the asynchronous offer pipeline.

5 Collections engine

6 Templates and product catalogue

7 Offer pipeline and automation

5 Collections engine

A collection is a named content type. Every collection owns exactly one physical table whose columns are created, renamed and dropped through the API while the system is running. Offers, customers' lookups, insurer credentials, templates and every product dimension are collections.

5.1 Meta-model

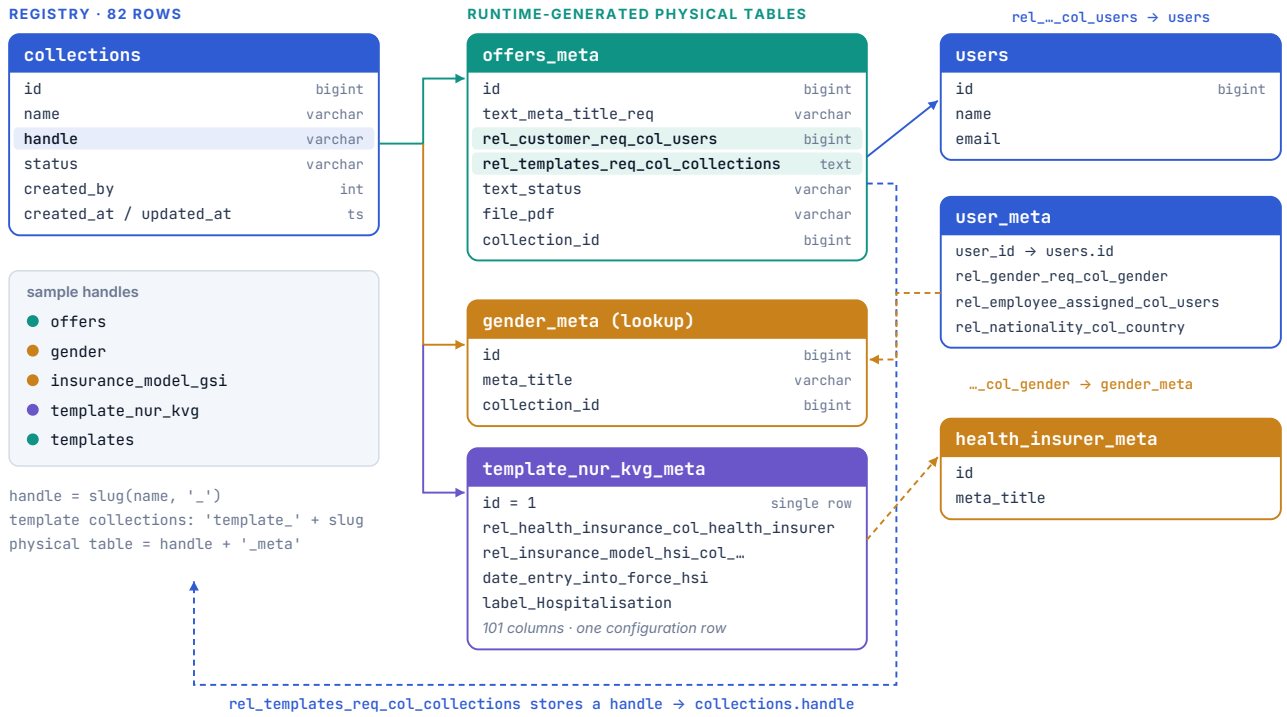


Figure 6 The collections registry and the physical tables it governs. Handles map one-to-one to `{handle}_meta` tables; relational columns point to other tables by naming convention rather than by foreign key.

The registry table `collections` holds 82 rows at export. Each row has a human name and a **handle** derived from it with `Str::slug(name, '_')`; template collections receive the prefix `template_`. The physical table name is always `handle + '_meta'`. Controllers never use Eloquent models for these tables — they resolve the table name at request time and use `DB::table()` and `Schema::` directly.

The special handle `user` is redirected to the user profile table: `validate_collection('user')` resolves to the `User` model, so the field builder can add profile fields to `user_meta` exactly like it adds fields to any collection.

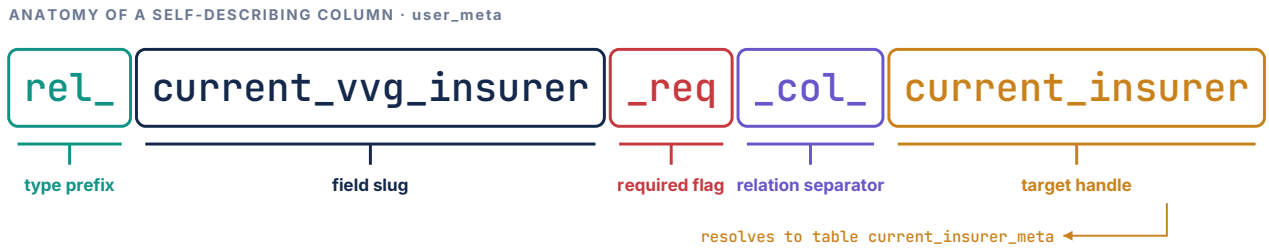
Collection life cycle

OPERATION	ENDPOINT	DATABASE EFFECT
Create	`POST` `collections/create`	Validates unique name; inserts registry row; `Schema::create` builds `{handle}_meta` with `id`, `meta_title`, `collection_id` (FK → collections, *set null*) and timestamps. If DDL fails the registry row is deleted.
Rename	`PUT` `collections/edit/{handle}`	Updates name and status; the handle is immutable so the physical table never moves.

OPERATION	ENDPOINT	DATABASE EFFECT
Delete	`DELETE` `collections/delete/{handle}`	`DROP TABLE {handle}_meta` then deletes the registry row. Relations pointing at the dropped table silently resolve to raw ids afterwards.
List	`GET` `collections/all`, `…/all/templates`	Registry joined with creator name; non-admins see only collections they created; the templates variant filters handles beginning with `template_`.
Read entries	`GET` `collections/show/{handle}`	All rows newest first, every relational cell expanded (section 5.4), plus the column list for the grid.

5.2 The column grammar

Every layer of the system agrees on one convention: the column name carries its own type and semantics. The back end reads it to choose SQL types and resolve relations; the front end reads it to choose an input widget and to enforce required fields; the field builder writes it.



COLUMN NAME	SQL TYPE	RENDERED AS	SEMANTICS
`text_meta_title_req`	`VARCHAR(255)`	text input	required
`textarea_template_description`	`LONGTEXT`	multi-line editor	
`date_entry_into_force_dasi`	`DATE`	date picker	
`label_Hospitalisation_lump-sum`	`VARCHAR(1)`	section heading	presentational
`file_pdf`	`VARCHAR(255)`	upload widget	stores path
`rel_gender_req_col_gender`	`VARCHAR(255)`	dropdown ← gender_meta	required
`rel_created_by_col_users`	`VARCHAR / INT`	dropdown ← users	ownership
`rel_choose_blueprints_req_col_collections`	`VARCHAR(255)`	dropdown ← template_* handles	required

Figure 7 Anatomy of a relational column and the full set of prefixes in use, with the SQL type the field builder creates and the widget the form renders.

Formal grammar

```
# column names produced and understood by TrustCore
column      ::= relational | typed | system
relational  ::= "rel_" slug [ "_req" ] "_col_" target
target      ::= "users" | "collections" | "boolean" | handle      # handle → {handle}_meta
typed       ::= prefix slug [ "_req" ] [ suffix ]
prefix      ::= "text_" | "textarea_" | "date_" | "label_" | "boolean_" | "status_" | "enum_"
              | "file_" | "files_" | "image_" | "images_" | "upload_" | "uploads_"
system      ::= "id" | "meta_title" | "collection_id" | "created_at" | "updated_at" | "user_id"
required(c) ::= contains(c, "_req")                                  # checked anywhere in the name
```

Field types offered by the builder

FIELD_TYPE	COLUMN PREFIX	SQL TYPE	FRONT-END RENDERING AND NOTES
`text`	`text_`	VARCHAR(255) NULL	Single-line input
`textarea`	`textarea_`	LONGTEXT NULL	Multi-line editor
`date`	`date_`	DATE NULL	Date picker component
`file`	`file_`	VARCHAR(255) NULL	Upload widget; stores the public-disk path written by `handleFileUploads()`
`boolean`	`boolean_`	TINYINT(1) NULL	Toggle. Product switches in templates use `rel_… _col_boolean` instead, pointing at the two-row `boolean_meta` lookup.
`status`	`status_`	ENUM('publish','draft')	Publication state
`enum`	`enum_`	ENUM('Active','Inactive') + index	Fixed two-value selector
`label`	`label_`	VARCHAR(1) NULL	Not data: renders the remainder of the name as a section heading inside long forms
`relational`	`rel_ … _col_ …`	VARCHAR(255) NULL	Dropdown filled from the target; `link_type` = *user*, *collection* (with `collection_id`) or *templates_collection*. No foreign key is created.

5.3 Field management

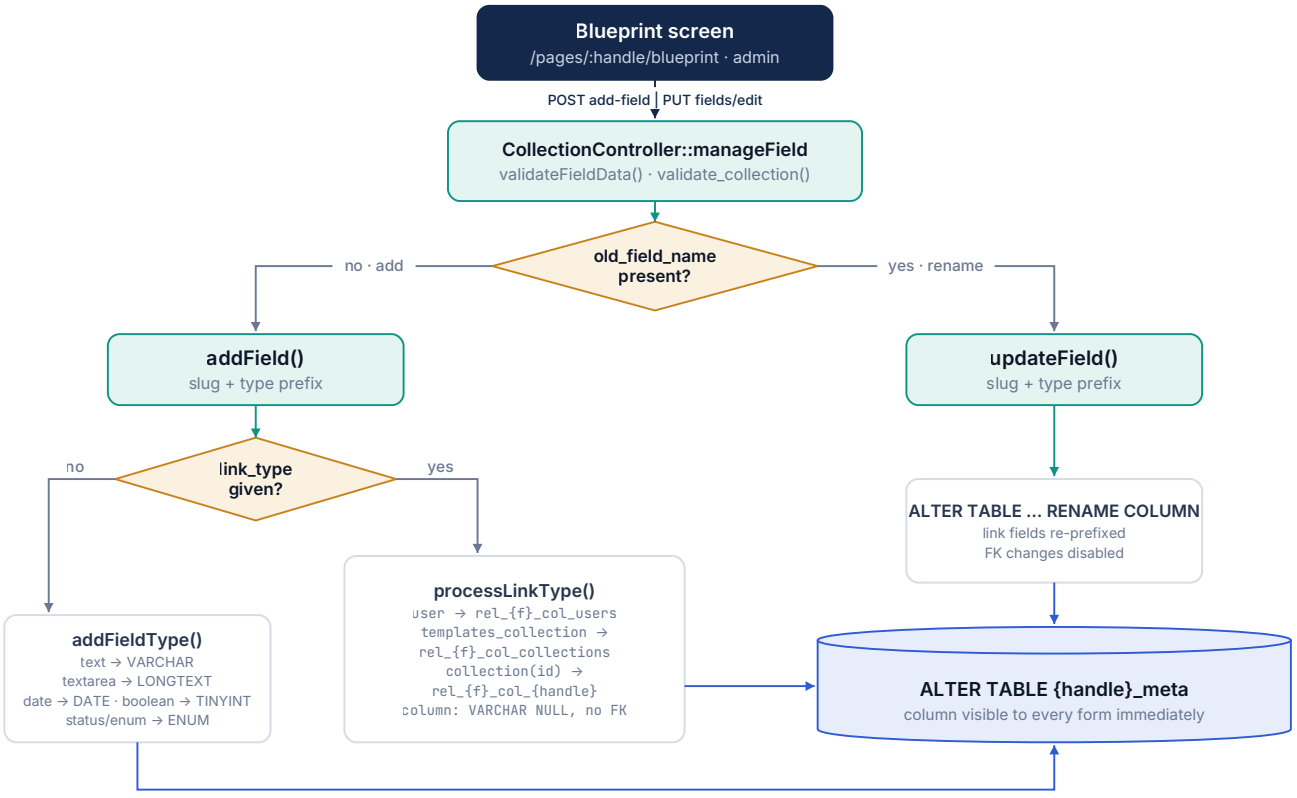


Figure 8 Decision flow of `CollectionController::manageField()` for adding and renaming fields.

Fields are added and renamed through a single entry point. The builder slugs the requested name, prefixes it with its type (relational fields receive their `rel_…_col_…` wrapper inside `processLinkType()`) and issues an `ALTER TABLE` on the collection's table. Renames are implemented as `RENAME COLUMN`; changing a field's type after creation is not supported — the type prefix changes but the SQL type does not. Deletion (`fields/delete`) looks up and drops any foreign-key constraint on the column through `INFORMATION_SCHEMA.KEY_COLUMN_USAGE`, then drops the column.

> **DESIGN CONSEQUENCE**
> Because relational fields are plain strings without foreign keys, referential integrity is maintained only by convention. Of the 74 foreign keys present in the database, 67 are the `collection_id → collections` keys created with each table; the remaining 7 belong to static tables (`user_meta`, `user_roles`, `contracts_meta`).

5.4 Relation resolution

Whenever entries are read, every relational cell is replaced by a structure containing the selected record and the full option list for its dropdown. The work is done by the trait `GetRelationalFieldDetail`, shared by three controllers.

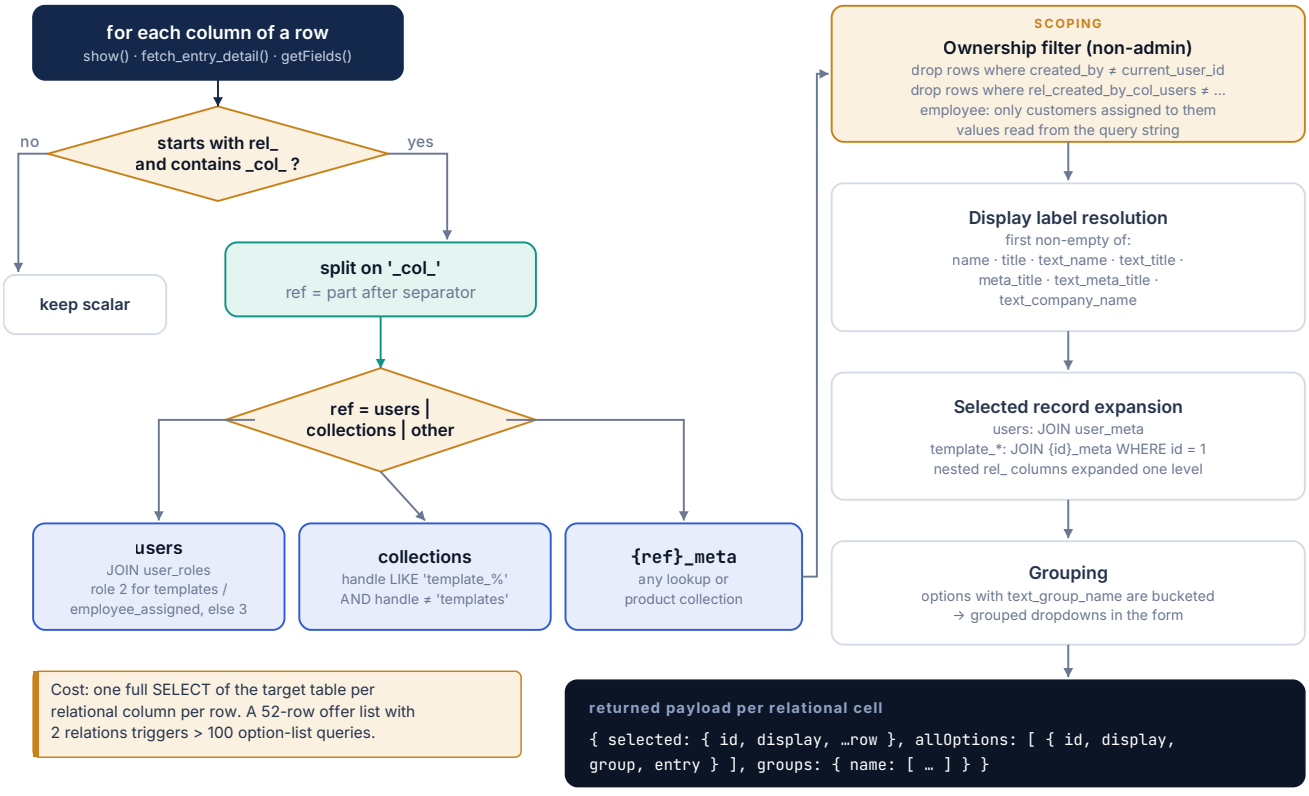


Figure 9 Relation resolution algorithm. The same function serves list views, single-entry views and empty forms (where only the option list is needed).

Resolution rules

- **Target `users`.** Options are users joined to `user_roles`: role 2 (employees) when the field is an *employee_assigned* field or the collection is `templates`; role 3 (customers) otherwise. Non-admins additionally see only customers assigned to them.

- **Target `collections`.** Options are registry rows whose handle starts with `template_` (excluding `templates` itself); the option id is the handle, not the numeric id. The selected template is expanded by joining its own table's row 1.
- **Any other target.** Options come from `{target}_meta`. If a lookup row has `text_group_name`, options are also bucketed into named groups for grouped dropdowns (used by coverage levels and waiting periods).
- **Display label.** The first non-empty of `name, title, text_name, text_title, meta_title, text_meta_title, text_company_name`.
- **Nested relations.** Relational columns of the selected record are expanded one further level.

The approach is flexible but expensive: each relational cell triggers a full read of its target table. Grids with many rows and several relational columns therefore issue hundreds of queries per request; caching option lists per request would remove most of that cost.

5.5 Entry operations

OPERATION	ENDPOINT	BEHAVIOUR
Form schema	GET `{handle}/get-fields`	Column list of the table with option lists for relational columns, plus all collections (id, name) for the builder.
Create	POST `{handle}/entry/create`	Every request key except uploaded files is inserted as a column value. Uploads are stored on the public disk as `uploads/{time}_{original name}`. Special cases: *templates* and *insurance-companies* set ownership; *offers* forces status and triggers the flow; *templates* also clones a template table (chapter 6).
Read	GET `{handle}/entry-detail/{id}`	Expanded entry plus the translation dictionary; `?cleandata=1` strips `_req` from keys for external consumers such as the robot.
Update	PUT `{handle}/entry-update/{id}`	Same mass-assignment model as create; non-admins cannot change `rel_created_by_col_users`; returns 404 if no row changed.
Delete	DELETE `{handle}/entry-delete/{id}`	Hard delete; no soft-delete column exists anywhere in the dynamic schema.

> **MASS ASSIGNMENT BY DESIGN**
> Create and update write whatever keys the client sends. Unknown keys fail with a database error (returned verbatim), but any existing column — including status, ownership and file paths — can be set by any authenticated caller. A per-collection allow-list derived from the column grammar would close this without losing flexibility.

6 Templates and product catalogue

Templates encode the Swiss health and supplemental insurance catalogue. A template is a collection whose table holds a single configuration row with up to 129 fields describing which product lines an offer includes and how each is configured.

6.1 Template anatomy

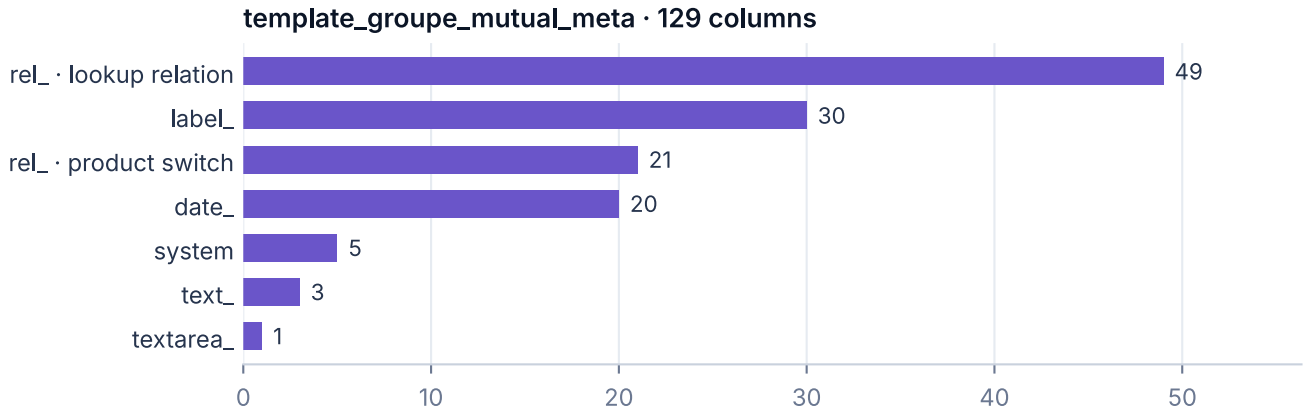


Figure 10 Column composition of the largest template. Lookup relations dominate; product switches are boolean relations; labels structure the form into sections.

Template tables are wide and shallow: 16 template tables hold 1,746 of the database's 2,206 columns but only one row each. The row with `id = 1` is the configuration; the form for a template is therefore always `/pages/template_{name}/entry/1`. Four templates are in productive use (*Groupe Mutuel Template 1*, *Group Mutuel Template*, *Group Mutuel Template 2.0* and *Nur KVG*); twelve are test templates (`template_test_*`).

6.2 Product catalogue

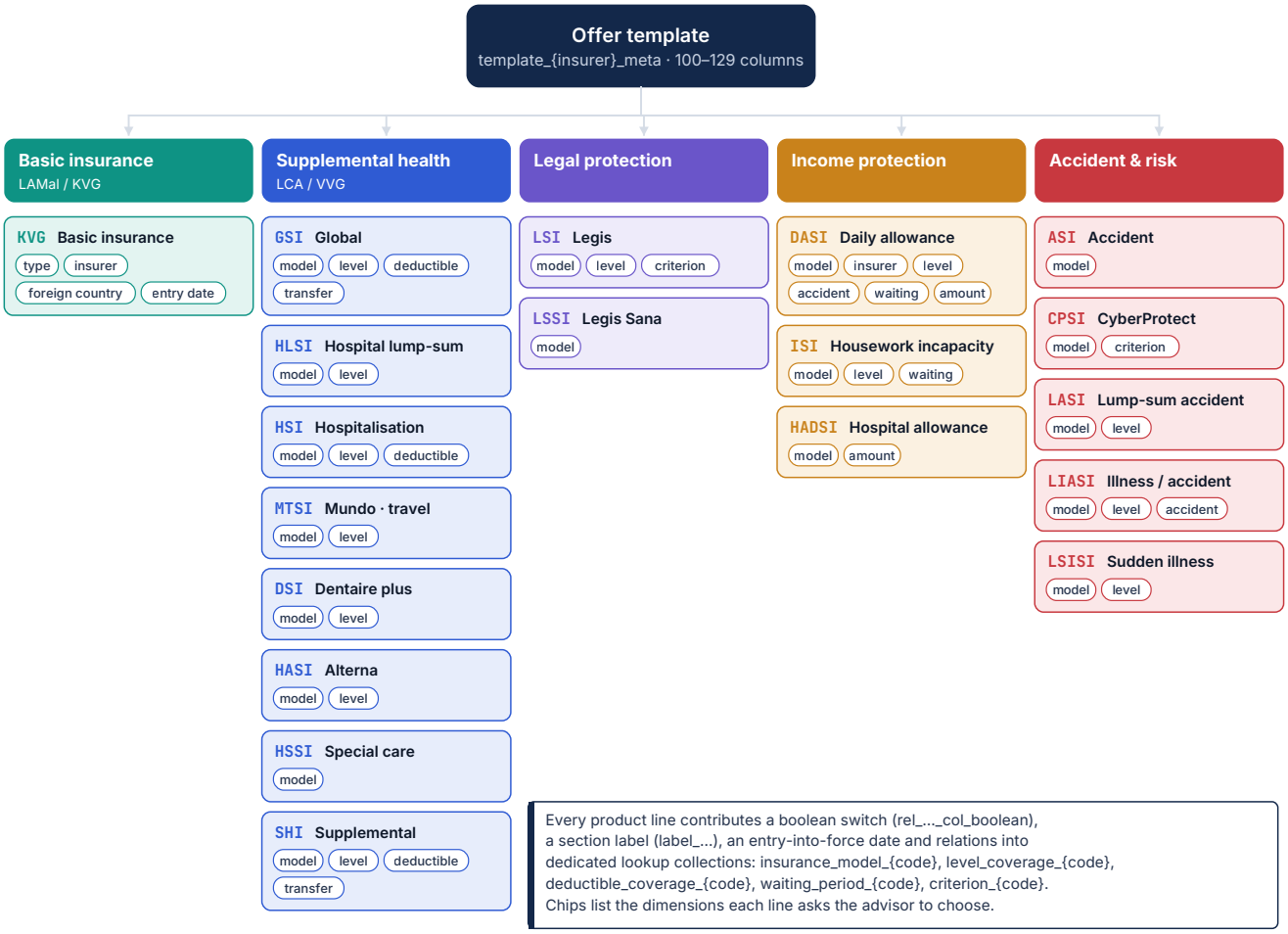


Figure 11 Product lines encoded in the Groupe Mutuel template, grouped into five families. Chips show the dimensions an advisor configures for each line.

CODE	PRODUCT LINE	FAMILY	LOOKUP COLLECTIONS USED
KVG	Basic insurance (LAMal / KVG)	Basic	insurance_type, health_insurer, country
GSI	Global	Supplemental	insurance_model_gsi, level_coverage_gsi, deductible_coverage, boolean (facilitated transfer)
HLSI	Hospitalisation lump-sum	Supplemental	insurance_model_hlsi, level_coverage_hlsi
HSI	Hospitalisation	Supplemental	insurance_model_hsi, level_coverage_hsi, deductible_coverage_hsi
MTSI	Mundo (travel)	Supplemental	insurance_model_mtsi, level_coverage_mtsi
DSI	Dentaire plus (dental)	Supplemental	insurance_model_dsi, level_coverage_dsi
HASI	Alterna (alternative medicine)	Supplemental	insurance_model_hasi, level_coverage_mtsi
HSSI	Special care	Supplemental	insurance_model_hssi
SHI	Supplemental health insurance	Supplemental	insurance_model_shi, level_coverage_shi, deductible_coverage_shi, boolean (facilitated transfer)
LSI	Legis (legal protection)	Legal	insurance_model_lsi, level_coverage_lsi, criterion_lsi
LSSI	Legis Sana	Legal	insurance_model_lssi

CODE	PRODUCT LINE	FAMILY	LOOKUP COLLECTIONS USED
DASI	Daily allowance	Income	insurance_model_dasi, health_insurer_dasi, level_coverage_dasi, accident_coverage_dasi, waiting_period_dasi, daily amount
ISI	Incapacity for housework	Income	insurance_model_isi, level_coverage_isi, waiting_period_isi
HADSI	Hospitalisation allowance	Income	insurance_model_hadsi, daily amount
ASI	Accident insurance	Accident & risk	insurance_model_asi
CPSI	CyberProtect (internet risks and disputes)	Accident & risk	insurance_model_cpsi, criterion_cpsi
LASI	Lump-sum on disability or death (accident)	Accident & risk	insurance_model_lasi, level_coverage_lasi
LIASI	Lump-sum illness / accident	Accident & risk	insurance_model_liasi, level_coverage_liasi, accident_coverage
LSISI	Accident or sudden illness	Accident & risk	insurance_model_lsisi, level_coverage_lsisi

The template also carries customer-context fields shared by all lines — payment method, invoicing, occupational status (required), franchise data, the insurer company it belongs to (`rel_company_col_insurance_companies`) and a free-text description.

6.3 Creating a template

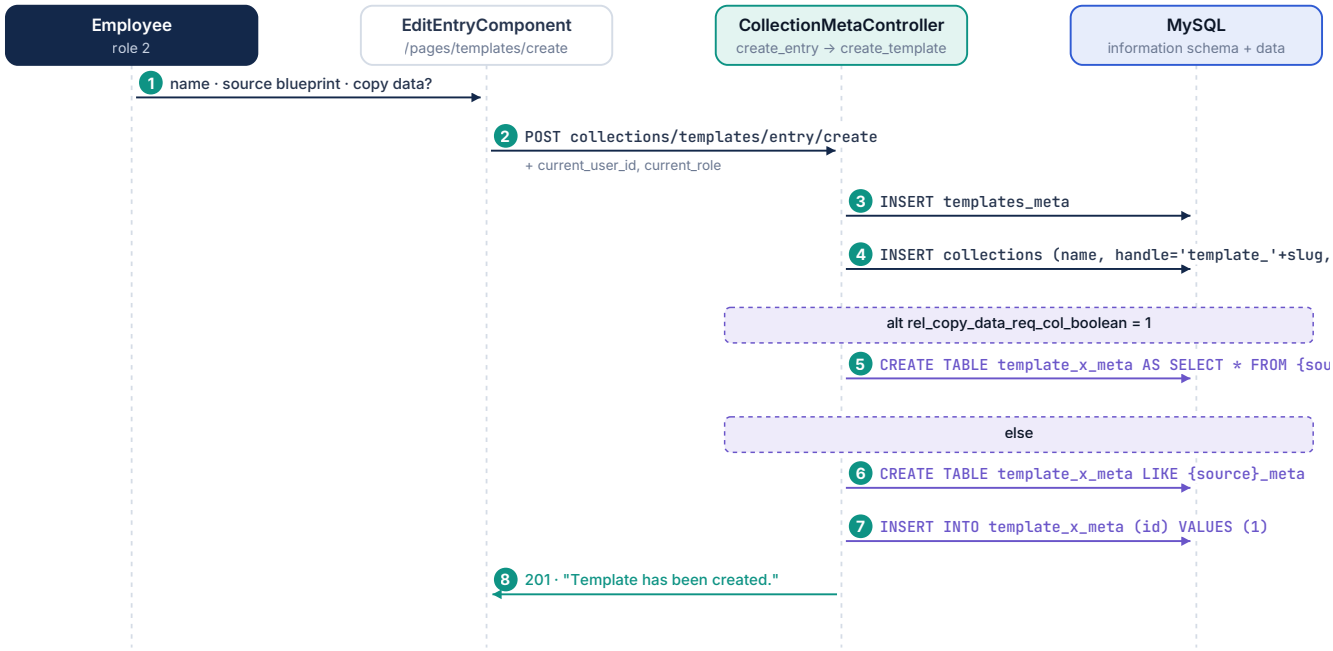


Figure 12 Template creation. The new template either copies the source blueprint's configuration row or starts from an empty row with the same structure.

Creating a template is an entry in the `templates` collection with three fields: a title, a source blueprint (`rel_choose_blueprints_req_col_collections` — the handle of an existing template) and a copy flag (`rel_copy_data_req_col_boolean`). After inserting the entry, `create_template()` registers a new collection `template_{slug(title)}` owned by the creator and clones the source table with either `CREATE TABLE … AS SELECT` (structure and data) or `CREATE TABLE … LIKE` plus an empty row 1.

STRUCTURAL CAVEATS

- `CREATE TABLE … AS SELECT` copies data and column types but not the primary key, auto-increment or indexes; copied templates therefore lack a primary key.
- The source handle is concatenated into the SQL statement without validation (risk R04).
- Templates are not versioned. An offer stores only the template handle; the robot reads the template's row at processing time, so editing a template after an offer was submitted changes what that offer will be built from.

6.4 Lookup collections

Sixty-four tables hold the option lists that templates and profiles refer to. Most have the minimal shape `id, meta_title, collection_id, timestamps`; coverage levels and waiting periods add `text_group_name` for grouped dropdowns. Appendix C lists every lookup table with its row count.

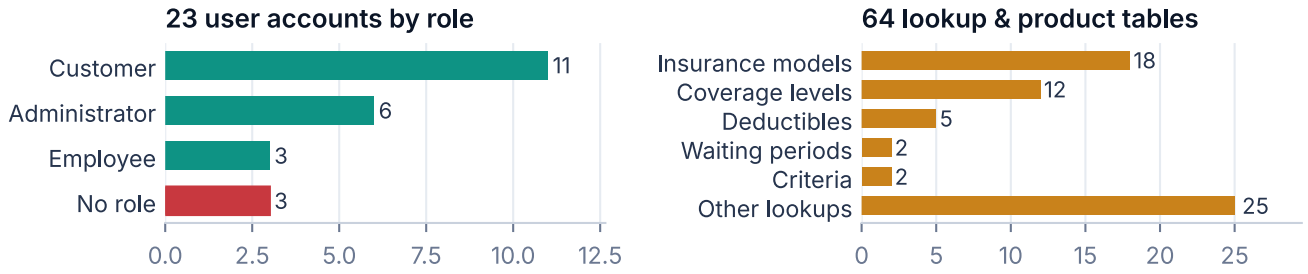


Figure 13 Left: user accounts by role at export. Right: composition of the 64 lookup and product tables.

7 Offer pipeline and automation

An offer is created synchronously and completed asynchronously. Laravel records the request and triggers a Power Automate cloud flow; an unattended desktop robot does the insurer-facing work and reports back through two webhooks.

7.1 End-to-end sequence

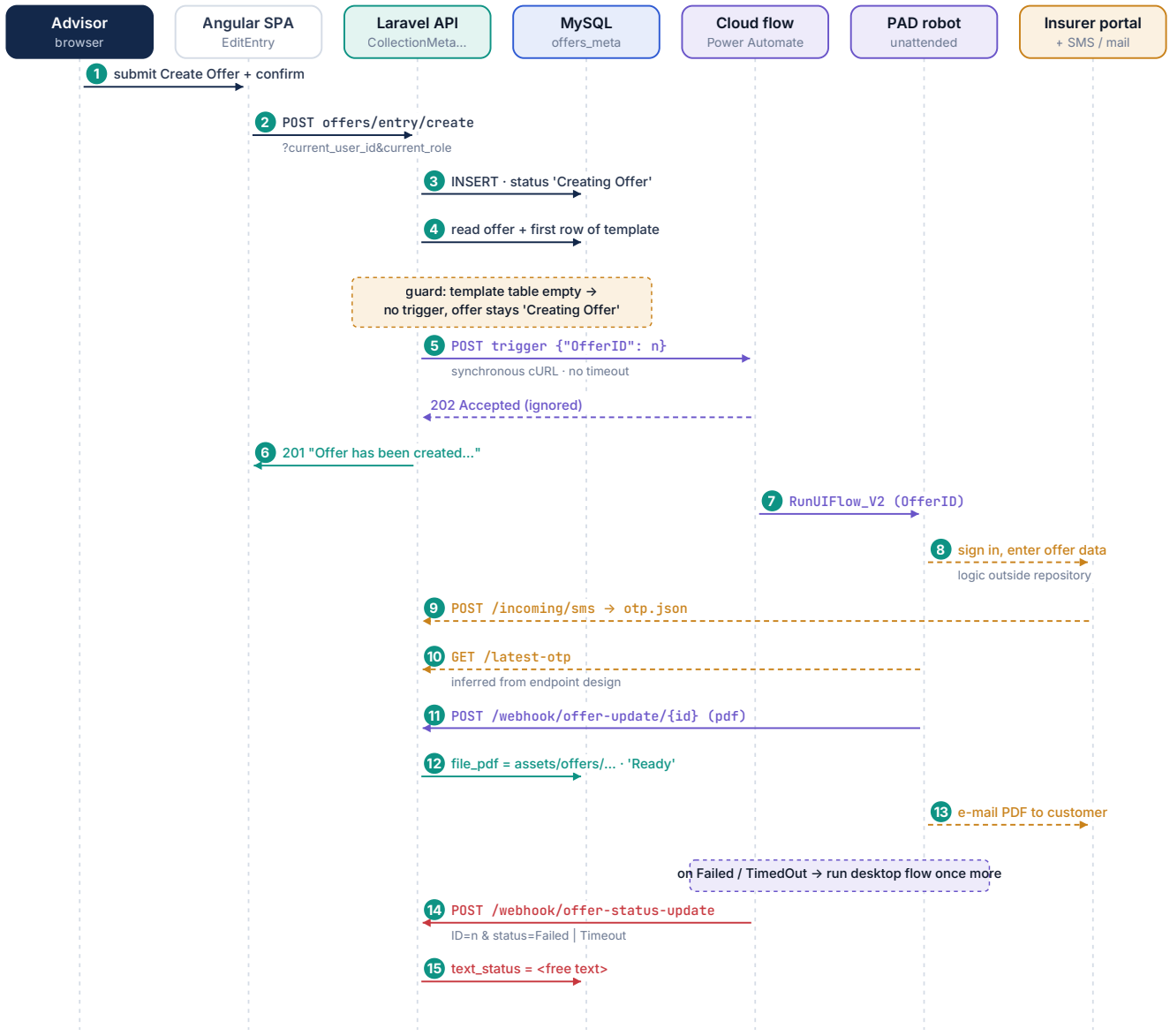


Figure 14 Offer creation from submission to PDF. Solid arrows are implemented in the TrustCore code base; dashed arrows run inside the automation and are inferred from the endpoints it calls.

Step reference

#	STEP	IMPLEMENTATION DETAIL
1–2	Submit	The generic entry form validates `_req` fields client-side, asks for confirmation and posts to `collections/offers/entry/create` with the user id and role in the query string.

#	STEP	IMPLEMENTATION DETAIL
3	Persist	`create_entry()` inserts into `offers_meta`, forcing `text_status = 'Creating Offer'`.
4	Guard	`createOffer()` reads the offer and the first row of `{template}_meta`. If the template table is empty, nothing is triggered and nothing is reported.
5	Trigger	cURL POST of `{"OfferID": n}` to the Logic Apps HTTP trigger (signed URL, hard-coded). `CURLOPT_TIMEOUT = 0`: the PHP request waits as long as the trigger takes; the response is ignored.
6	Acknowledge	HTTP 201 with the message “Offer has been created. You'll receive an email with offer PDF”.
7	Robot run	Cloud flow action `RunUIFlow_V2`, run mode *unattended*, input `OfferID`.
8–10	Portal work	The robot operates the insurer portal. A six-digit code arriving by SMS is captured by `/incoming/sms` and is available on `/latest-otp`; the robot is the only plausible consumer.
11–12	Deliver	Multipart POST of field `pdf` to `/webhook/offer-update/{id}`; Laravel saves `assets/offers/offer_{name}_{time}.{ext}`, sets `file_pdf` and status *Ready*.
13	Notify	Customer e-mail with the PDF, sent by the automation.
14–15	Failure	On failure or timeout of the retry, the cloud flow posts `ID` and `status` to `/webhook/offer-status-update`; the text is stored verbatim.

7.2 Offer states

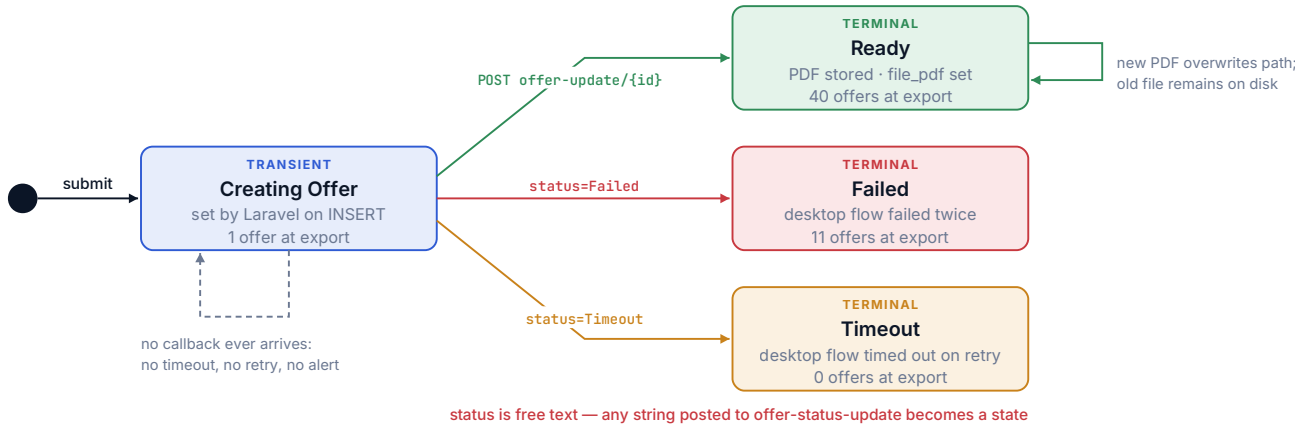


Figure 15 Offer state machine with the distribution observed at export.

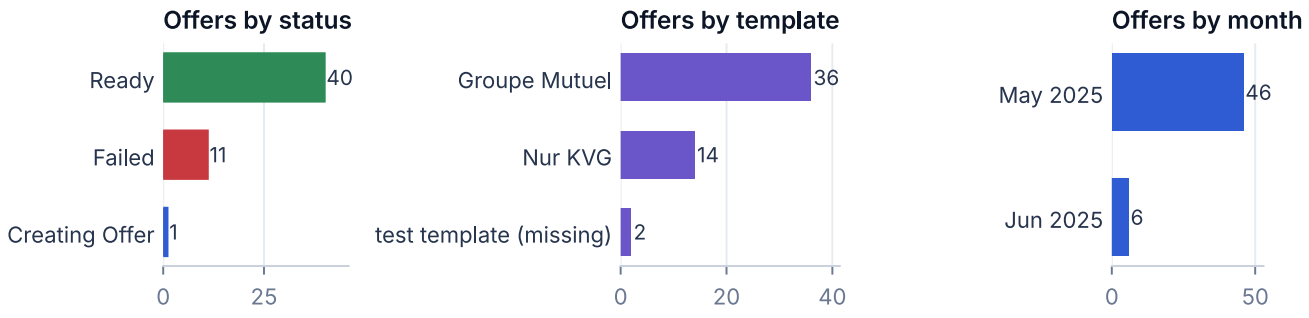


Figure 16 Offers at export by status, by template and by creation month.

Status is free text. Laravel writes *Creating Offer* and *Ready*; the cloud flow writes *Failed* and *Timeout*; any other string posted to the status webhook becomes a new de-facto state that the dashboard displays unchanged. Failure rate at export: 11 of 52 offers (21 %). Every offer was created in May and June 2025; the Group Mutuel Template accounts for 36 of them. Two offers reference the handle `template_test_template`, which does not exist in the registry.

7.3 Power Automate cloud flow

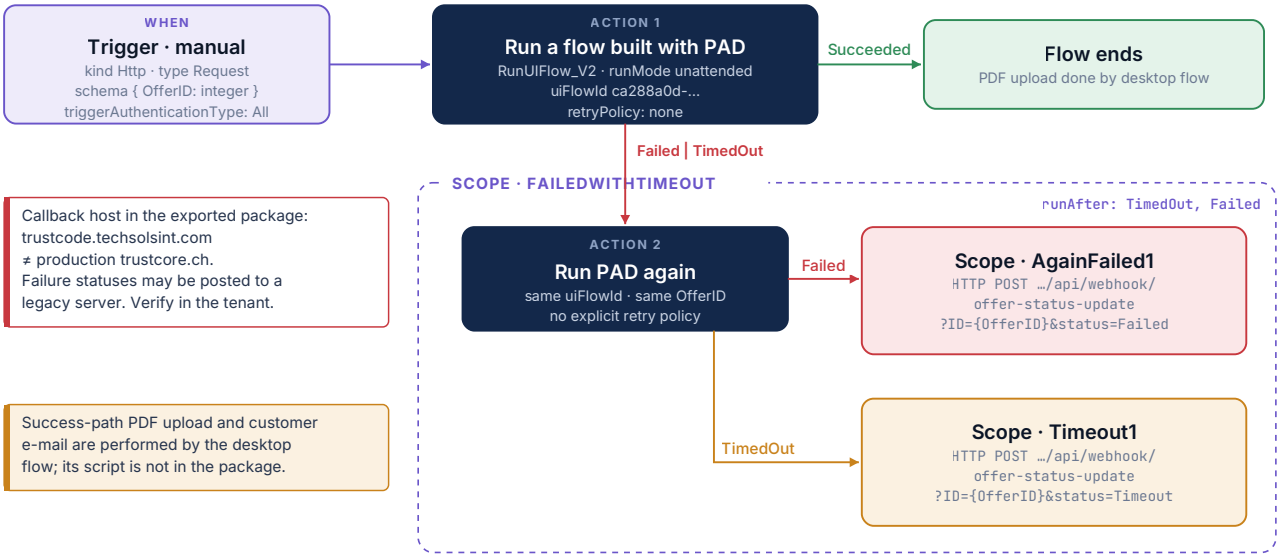


Figure 17 Structure of the exported cloud flow *PAD* (definition schema 2016-06-01). The flow retries the desktop robot once and reports the outcome only on failure.

ELEMENT	CONFIGURATION IN THE EXPORTED PACKAGE
Package	`PAD_20261022162223.zip`, created 22 Oct 2026; flow display name *PAD*; client created 13 Sep 2025, last modified 1 May 2025 (migrated from a Logic Apps shadow workflow)
Trigger	`manual`, kind *Http*, body schema `{ OfferID: integer }`, `triggerAuthenticationType: All`
Action 1	`Run_a_flow_built_with_Power_Automate_for_desktop` — `RunUIFlow_V2`, `uiFlowId ca288a0d-5800-f011-bae3-002248ca7e1b`, unattended, `retryPolicy: none`
Scope FailedWithTimeout	runAfter *TimedOut* or *Failed*; runs the same desktop flow a second time
Scope AgainFailed1	after a second failure: `POST …/api/webhook/offer-status-update?ID={OfferID}&status=Failed`
Scope Timeout1	after a second timeout: same endpoint with `status=Timeout`
Connection	`shared_uiflow` (“Desktop flows”), embedded connection reference, unattended
Failure alerts	`failureAlertSubscription: true` in metadata, `flowFailureAlertSubscribed: false`

CALLBACK HOST MISMATCH

Both failure callbacks in the exported flow target `https://trustcode.techsolsint.com/trustcore-backend/public/api/…`, not `trustcore.ch`. If the deployed flow matches the export, failure statuses are being posted to the legacy server and production offers stay in *Creating Offer*. In addition, the workflow id in Laravel's trigger URL differs from the exported flow's id. Both must be confirmed in the Power Automate tenant.

7.4 SMS one-time-code relay

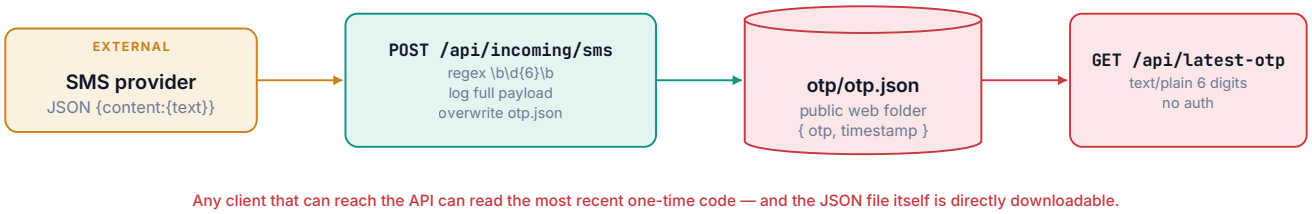


Figure 18 OTP relay. The latest six-digit code is written to a JSON file inside the public web folder and served without authentication.

`SmsController::receiveSms` logs every inbound payload in full, extracts the first stand-alone six-digit number from `content.text` and overwrites `public/otp/otp.json` with the code and a timestamp. `getLatestOtp` returns the code as plain text. There is no binding between a code and the offer or robot run that requested it, so concurrent offers can consume each other's codes.

7.5 Failure modes

FAILURE	OBSERVABLE EFFECT	RECOMMENDED CONTROL
Template table empty	Offer remains *Creating Offer*; no error	Validate before insert; return 422
Trigger slow or unreachable	PHP request blocks (no timeout); user sees a hung form	Dispatch a queued job with timeout and retries
Robot never calls back	Offer stuck indefinitely	Scheduled sweeper: mark *Timeout* after N minutes and alert
Failure callback to wrong host	Production offer never leaves *Creating Offer*	Callback base URL as a flow parameter per environment
Concurrent OTPs	Wrong code used; portal sign-in fails	Store codes per run id with expiry
Template edited after submission	PDF reflects new configuration	Snapshot the template row into the offer at submit time
Callback spoofing	Any client can set status or attach a file	Shared secret / HMAC header and file-type validation

PART THREE

III

Application Layers

How the Laravel API and the Angular application are organised, every endpoint they expose and consume, and how identity and permissions flow between them.

8	Back-end architecture
9	API reference
10	Front-end architecture
11	Identity and access control

8 Back-end architecture

The API is a Laravel 11 application in its slim configuration: one bootstrap file, route files for API, web and console, and eight controllers that hold nearly all business logic. Two traits provide shared behaviour; Eloquent models exist only for the static tables.

8.1 Request life cycle

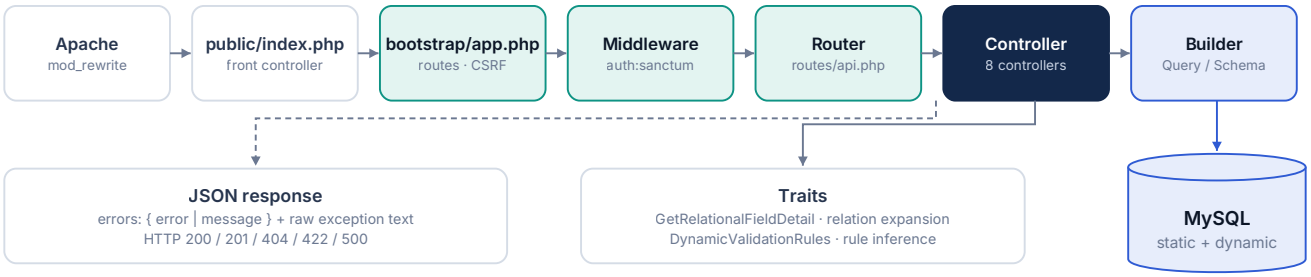


Figure 19 Path of an API request through the framework. Every controller action wraps its body in try/catch and returns JSON, including exception text.

`bootstrap/app.php` registers the web, API, console and health routes and exempts `api/*` from CSRF verification. The stateful-frontend middleware for Sanctum is commented out, so the API is purely token-based: authenticated routes are grouped under `auth:sanctum`, which resolves the bearer token against `personal_access_tokens`. There are no policies, gates or custom middleware for authorisation; the only custom middleware class (`EnsureEmailIsVerified`) is not registered.

8.2 Component structure

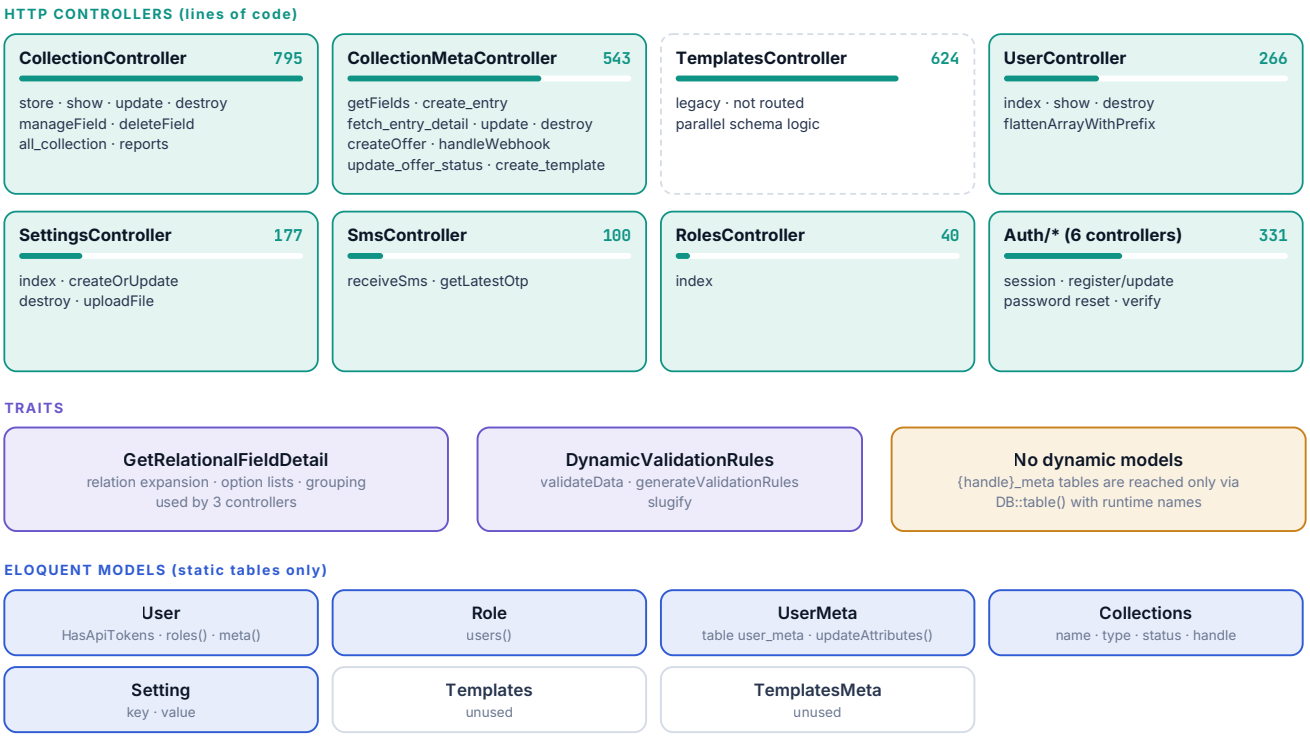


Figure 20 Controllers, traits and models with their size and responsibilities. TemplatesController is a parallel, unrouted implementation retained in the code base.

Controller responsibilities

CONTROLLER	LOC	PUBLIC ACTIONS AND NOTABLE PRIVATE HELPERS
`CollectionController`	795	`store`, `all_collection`, `show`, `update`, `destroy`, `manageField`, `deleteField`, `reports`; helpers `validateFieldData`, `addField`, `updateField`, `processLinkType`, `getFieldTypes`, `getForeignKeyName`
`CollectionMetaController`	543	`getFields`, `create_entry`, `fetch_entry_detail`, `update`, `destroy`, `handleWebhook`, `update_offer_status`; helpers `create_template`, `createOffer`, `handleFileUploads`
`TemplatesController`	624	Earlier template-specific copy of the collection logic. Imported in `routes/api.php` but its only route is commented out.
`UserController`	266	`index` (role and assignment filters, flattened `roles_*` / `meta_*` columns with types), `show`, `destroy` (refuses self-deletion)
`SettingsController`	177	Key–value site settings; logo upload to `storage/logos`; any key is accepted and stored
`SmsController`	100	`receiveSms`, `getLatestOtp` (chapter 7)
`RolesController`	40	`index`: all roles plus column list
`Auth\*`	331	`AuthenticatedSessionController` (login issues a token, logout revokes the current one), `RegisteredUserController` (create *and* update user with profile fields), password reset link and new password, e-mail verification

Shared traits

`GetRelationalFieldDetail` implements relation resolution (section 5.4). It reads `$_REQUEST['current_user_id']` and `$_REQUEST['current_role']` directly from PHP super-globals to apply ownership filtering.
`DynamicValidationRules` infers validation rules from the submitted values themselves — numeric values become `integer`, strings `string`, arrays `array` — and therefore accepts any key; it also provides the `slugify()` used for template table names.

User registration and profile update

`RegisteredUserController::store` serves three routes (public web `/register`, `/api/user/register` and `/api/user/edit/{id}`). Keys prefixed `meta_` are routed to `user_meta`; the remaining keys update `users`. When first and last name are present the display name is composed from them. Role 3 is the default; `roles_id`, if present, replaces the user's roles. Profile keys are written only if a column with that name exists (`UserMeta::updateAttributes`).

8.3 Error handling conventions

STATUS	PRODUCED WHEN
`200 / 201`	Success. Create operations return 201 with `{message, id}`.
`404`	Unknown handle (`firstOrFail`), missing meta table, missing entry, or an update that changed no row.
`422`	Laravel validation failures (field management, settings, login).
`429`	Login throttled after five failed attempts per e-mail and IP.
`500`	Any exception. The body contains `"Database error: …"` or `"An unexpected error occurred: …"` with the raw exception message, including SQL fragments.

9 API reference

All business endpoints live under `/api`. Authenticated endpoints expect `Authorization: Bearer <token>`. The Angular client appends `current_user_id` and `current_role` to every collection and user call; those parameters are listed where the server reads them.

9.1 Conventions

ASPECT	CONVENTION
Base URL	Production `https://trustcore.ch/trustcore/trustcore-backend/public/api/` · local `http://127.0.0.1:8000/api/`
Formats	Request JSON or `multipart/form-data` (uploads); response JSON except `/latest-otp` (text/plain)
Access labels	TOKEN requires Sanctum token · GUEST public by design · OPEN public but should not be
Path parameter `{handle}`	Collection handle; slugged with `_` before lookup (so `insurance-companies` resolves to `insurance_companies`)

9.2 Authentication

VERB	PATH	ACCESS	REQUEST → RESPONSE
POST	`/login`	GUEST	`{email, password}` → `{message, user{…, roles[], meta}, token}`. Throttled 5 attempts / key. Updates `last_login`.
POST	`/logout`	TOKEN	Deletes the current token only.
POST	`/request-pass`	GUEST	`{email}` → broker sends reset link; `{status}` or 422.
POST	`/reset-password`	GUEST	`{token, email, password, password_confirmation}` → Laravel password broker.

9.3 Collections and schema

VERB	PATH	ACCESS	REQUEST → RESPONSE
GET	`/collections/all`	TOKEN	Reads role params → `{collection: [ … , created_by_name ]}`; 404 if empty
GET	`/collections/all/templates`	TOKEN	Same, filtered to `template_*`
POST	`/collections/create`	TOKEN	`{name, status?, is_template_collection?}` → 201 `{collection}`; creates table
GET	`/collections/show/{handle}`	TOKEN	→ `{columns[], data[]}` with expanded relations; employee filters for offers and insurance companies
PUT	`/collections/edit/{handle}`	TOKEN	`{name, status?}` → `{collection}`
DELETE	`/collections/delete/{handle}`	TOKEN	Drops `{handle}_meta` and the registry row
POST	`/collections/{handle}/add-field`	TOKEN	`{field_name, field_type, link_type?, collection_id?}` → ALTER TABLE ADD
PUT	`/collections/{handle}/fields/edit`	TOKEN	`{old_field_name, new_field_name, field_type?, link_type?}` → RENAME COLUMN
DELETE	`/collections/{handle}/fields/delete`	TOKEN	`{field_name}` → drop FK if any, DROP COLUMN
GET	`/collections/{handle}/get-fields`	TOKEN	→ `{collection, fields{col: '' | {allOptions}}, allCollections}`

9.4 Entries

VERB	PATH	ACCESS	REQUEST → RESPONSE
POST	`/collections/{handle}/entry/create`	TOKEN	Column values (+ files) → 201 `{message, id}` ; offers trigger the flow; templates clone a table
GET	`/collections/{handle}/entry-detail/{id}`	TOKEN	`?cleandata=1` optional → `{collection, entry, translations}`
PUT	`/collections/{handle}/entry-update/{id}`	TOKEN	Column values → `{message, collectionHandle, validatedData}`
DELETE	`/collections/{handle}/entry-delete/{id}`	TOKEN	→ `{message}` (200 also when nothing was found)

9.5 Users, roles, settings, reports

VERB	PATH	ACCESS	REQUEST → RESPONSE
GET	`/user/all`	TOKEN	`?role` , `?employee_assigned` , role params → `{data[], columns{name: type}}`
GET	`/user/detail[/{id}]`	TOKEN	Defaults to the token owner → flattened user
POST	`/user/register`	TOKEN	`{email, password, roles_id?, meta_*}` → `{message, user}`
POST	`/user/edit/{id}`	TOKEN	Same payload; password optional
DELETE	`/user/delete/{id}`	TOKEN	Detaches roles, deletes profile and user; 403 for self
GET	`/user/reports/all`	TOKEN	12-month report (chapter 13)
GET	`/roles`	TOKEN	→ `{data[], columns[]}`
GET	`/settings`	GUEST	→ `{settings{key: value}}` (branding before login)
POST	`/settings`	TOKEN	Any keys; `logo` file; `hidden_pages` array → all settings
DELETE	`/settings/{key}`	TOKEN	Deletes the key (and logo file)

9.6 Integration endpoints

VERB	PATH	ACCESS	REQUEST → RESPONSE
POST	`/webhook/offer-update/{offerId}`	OPEN	multipart `pdf` (any file) → `{message, offerID, offerCreated}` ; status *Ready*
POST	`/webhook/offer-status-update`	OPEN	`{ID, status}` (body or query) → `{message, offerID, status}`
POST	`/incoming/sms`	OPEN	`{content:{text}}` → `{status, otp}` or 400
GET	`/latest-otp`	OPEN	→ six digits as text/plain; 404 if none
GET	`/clear-cache`	OPEN	Runs cache, config, route and view clear

9.7 Session routes and framework endpoints

Outside `/api` , `routes/auth.php` registers the Breeze session routes `POST /register` , `/login` , `/forgot-password` , `/reset-password` , `/logout` , `GET /verify-email/{id}/{hash}` and `POST /email/verification-notification` . The Angular client does not use them. `GET /` returns the Laravel version and `GET /up` is the health probe. A copy of an older route set is kept in `public/routes-old/` — inside the web root.

9.8 Example exchange

```
# create an offer (as sent by the Angular client)
POST /api/collections/offers/entry/create?current_user_id=7&current_role=employee
Authorization: Bearer <sanctum-token>
Content-Type: application/json

{ "text_meta_title_req": "Family offer 2026",
  "rel_customer_req_col_users": 18,
  "rel_templates_req_col_collections": "template_nur_kvg" }

# → 201 Created
{ "message": "Offer has been created. You'll recieve an email with offer PDF",
  "id": 53 }
```

10 Front-end architecture

The Angular application is the ngx-admin starter with TrustCore's own modules grafted on. A small set of generic components renders every business screen from the API's column metadata.

10.1 Module and component structure

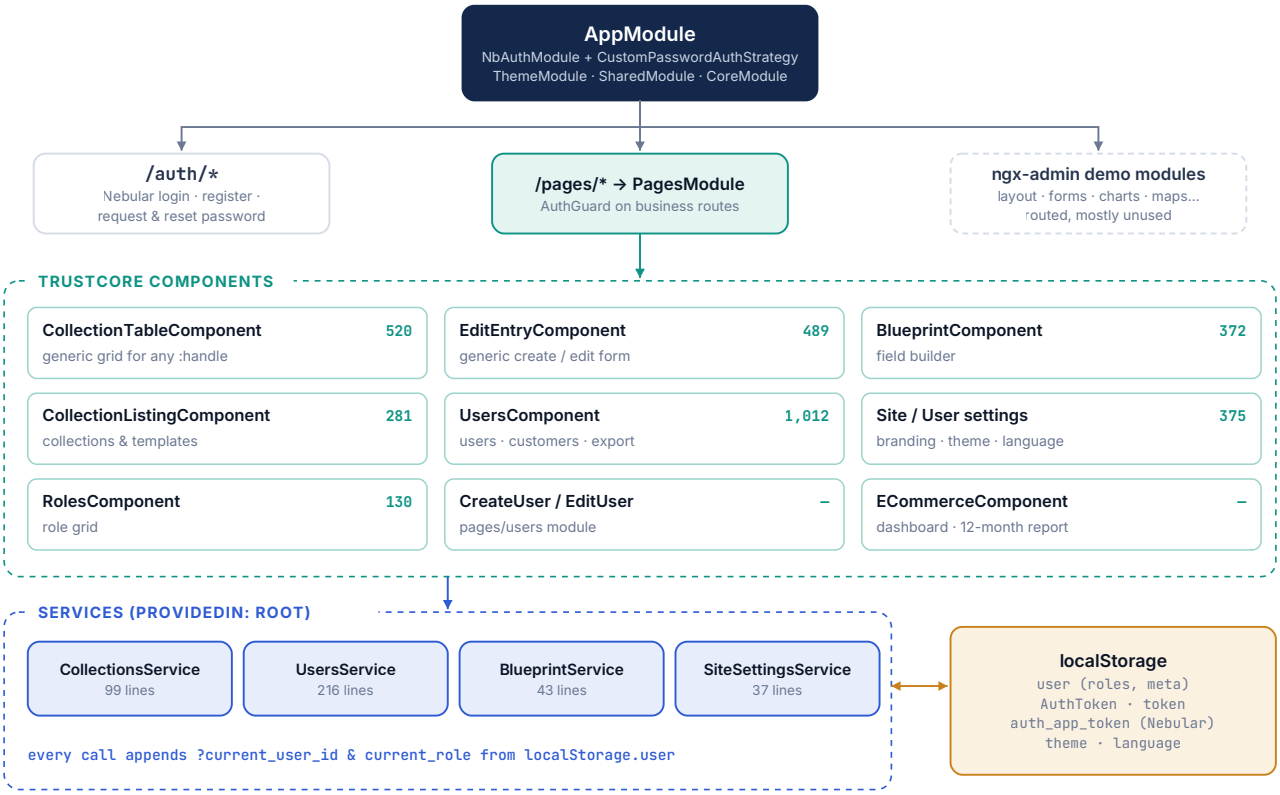


Figure 21 Application modules, TrustCore components (with line counts of their TypeScript) and services. Demo modules from the template remain routed.

10.2 Routing

PATH UNDER /PAGES	COMPONENT	PERMISSION DATA	PURPOSE
`dashboard`	ECommerceComponent	dashboard	Landing page with 12-month report
`collections/templates/all`	CollectionListingComponent	templates	Template list
`collections/all`	CollectionListingComponent	collections	All collections (admin)
`settings/site` · `settings/user`	Site- / UsersettingsComponent	settings	Branding · theme and language
`:handle`	CollectionTableComponent	:handle	Entry grid for any collection
`:handle/blueprint`	BlueprintComponent	blueprints	Field builder
`:handle/create` · `:handle/entry/:entryId` · `:handle/edit/:id`	EditEntryComponent	pagesCreate / pagesEdit	Generic form
`customers` · `reports`	CollectionTableComponent	users / reports	Unreachable — shadowed by `:handle`

PATH UNDER /PAGES	COMPONENT	PERMISSION DATA	PURPOSE
`users/list` · `users/list/all`	UsersComponent	users	Customers (employee) · all users (admin)
`user/profile[/create|/:id]`	UsersComponent	profile	Profile views
`roles/all` · `permissions`	Roles- / CollectionTableComponent	—	`permissions` has no guard
`password-reset/:handle`	UsersComponent	—	No guard

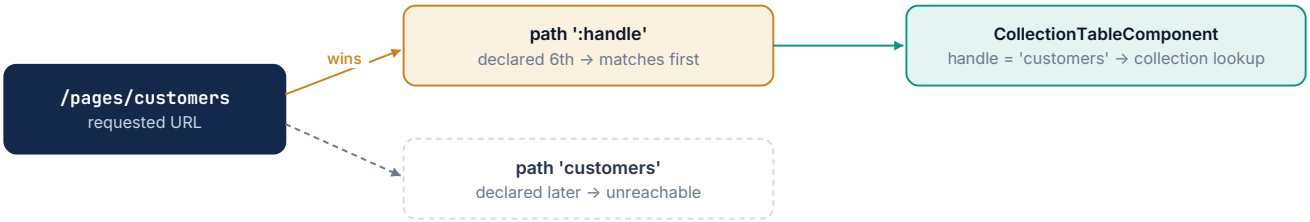


Figure 22 Route precedence. Angular matches in declaration order, so the explicit customers and reports routes are never reached.

10.3 The generic screens

CollectionTableComponent

Loads `collections/show/{handle}`, builds ng2-smart-table column definitions from the returned column list, hides system columns, renders relational cells through their resolved display value and offers create, edit and delete actions. For template collections an admin additionally sees a *Blueprint* button.

EditEntryComponent

Uses `get-fields` for new entries and `entry-detail` for existing ones, then classifies each key with `isTextField`, `isTextareaField`, `isDateField`, `isLabelField`, `isFileField` and `isDropdownField`. Labels for display are derived by stripping `rel_`, the `_col_…` suffix and `_req`, replacing `created_by` with “Assign to” and translating. Required fields (`_req`) are checked before submission. Non-admins never see `rel_created_by_col_users`.

BlueprintComponent

Lists the columns of a collection and lets an admin add, rename and delete fields, choosing type and — for relations — the link type (user, collection or template collection) and target collection. It is the UI for chapter 5's field management.

UsersComponent

The largest component (1,012 lines). It shows staff and customers in a smart table with flattened `roles_*` and `meta_*` columns, opens create and edit forms, and exports the current view to PDF (jsPDF + autotable) and Excel (SheetJS + file-saver).

10.4 Client-side state

LOCALSTORAGE KEY	WRITTEN BY	USED FOR
`user`	auth strategy on login	Role and id for menus, guard, and every API call's query string; language from `meta.settings`
`AuthToken` · `token`	auth strategy	Bearer header (`AuthToken`); `token` is also sent as the password-reset token

LOCALSTORAGE KEY	WRITTEN BY	USED FOR
`auth_app_token`	Nebular token service	Presence check in `AuthGuard`
theme · language	user settings screen	Nebular theme switching and translation lookup

10.5 Internationalisation

Menu labels and form labels are translated through `src/app/translations/translations.json`: 281 keys, each with `en`, `fr`, `de` and `it` values. The current language is read from the user's stored settings; English returns the key itself, other languages fall back to the key when no translation exists. The back end ships a copy of the dictionary in `resources/translations/` and returns it with every entry detail.

10.6 Build and toolchain

- Node 18 is required: the `postinstall` step runs `ngcc`, which fails on current Node releases.
- `npm install --legacy-peer-deps` is required because `@angular/google-maps@12` conflicts with Angular 15.
- `npm run build:prod` produces the AOT bundle whose API base URL comes from `environment.prod.ts`.
- Lint (tslint, stylelint), Karma unit tests and Protractor e2e configurations are inherited from ngx-admin; no TrustCore-specific tests exist.

11 Identity and access control

Authentication is sound and standard: Laravel Sanctum issues opaque bearer tokens after a rate-limited password check. Authorisation is the weak point — role decisions are made in the browser and in query parameters supplied by the browser.

11.1 Sign-in

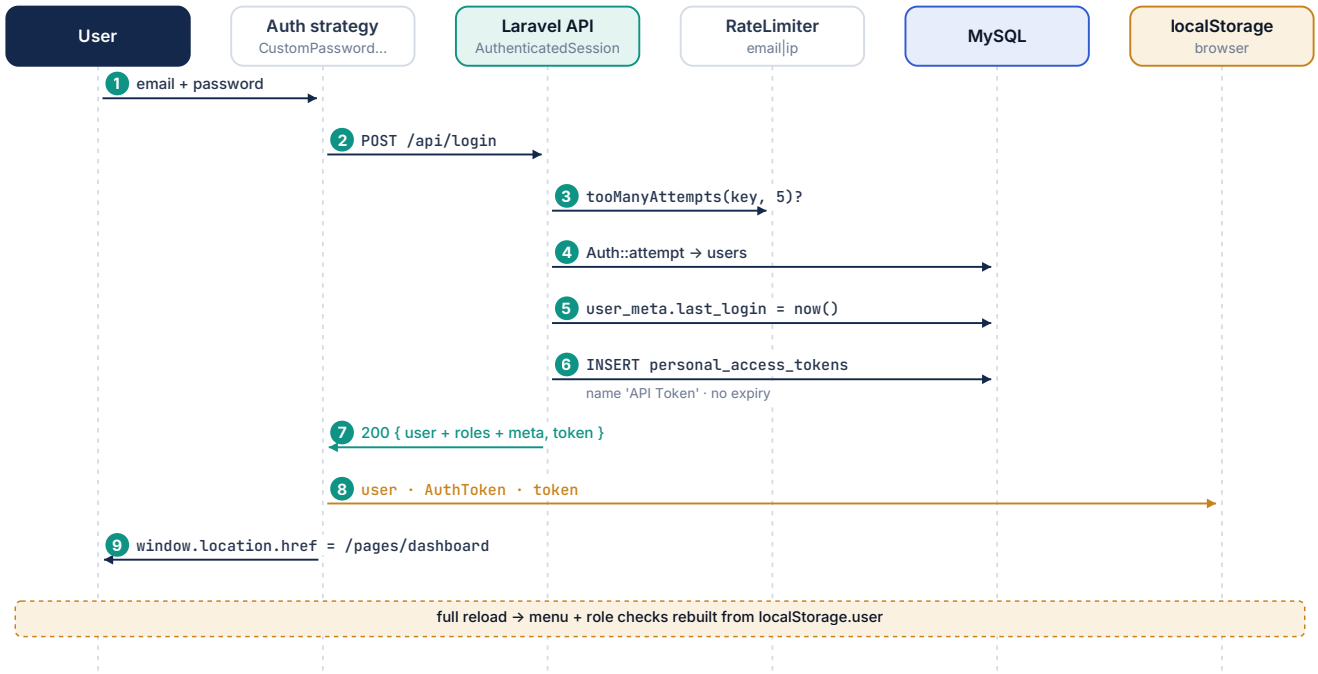


Figure 23 Sign-in sequence. The page reload at the end is deliberate: the menu is computed once, at module load, from localStorage.

Tokens are named *API Token*, carry the default `*` ability and never expire (`sanctum.expiration = null`). Logout revokes only the token in use; 811 tokens had accumulated in `personal_access_tokens` at export.

> **PASSWORD RESET FROM THE DASHBOARD**
> The custom strategy's `resetPassword()` sends the stored Sanctum token as the reset token and the e-mail of the user in localStorage. Laravel's password broker expects the token from the reset e-mail, so this path cannot succeed; the e-mailed link is the only working reset flow.

11.2 Role model

Roles are rows in `roles` (1 admin, 2 employee, 3 customer) linked through `user_roles`. The code treats the first role as the user's only role. Advisors are linked to customers through `user_meta.rel_employee_assigned_col_users`, the only enforced foreign key among the profile relations.

Permission matrix (as designed in the front end)

MENU AREA	ADMIN	EMPLOYEE	CUSTOMER
Dashboard	●	●	●
Offers — create and list	●	●	●
Templates — create and list	●	●	—
Company Credentials	●	●	—
Customers	—	●	—
Users (all accounts, role assignment)	●	—	—
Dashboard settings (language, theme)	●	●	●
Collections (raw content model)	●	—	—
Blueprint button on templates	●	—	—
Reports menu	listed, menu entry disabled	—	listed, menu entry disabled

Source: `accessControl()` in `pages/pages-menu.ts`. Each role's list is a single comma-joined string; `includes()` therefore performs substring matching (the code tests for `'insurance-companie'`).

11.3 Enforcement layers

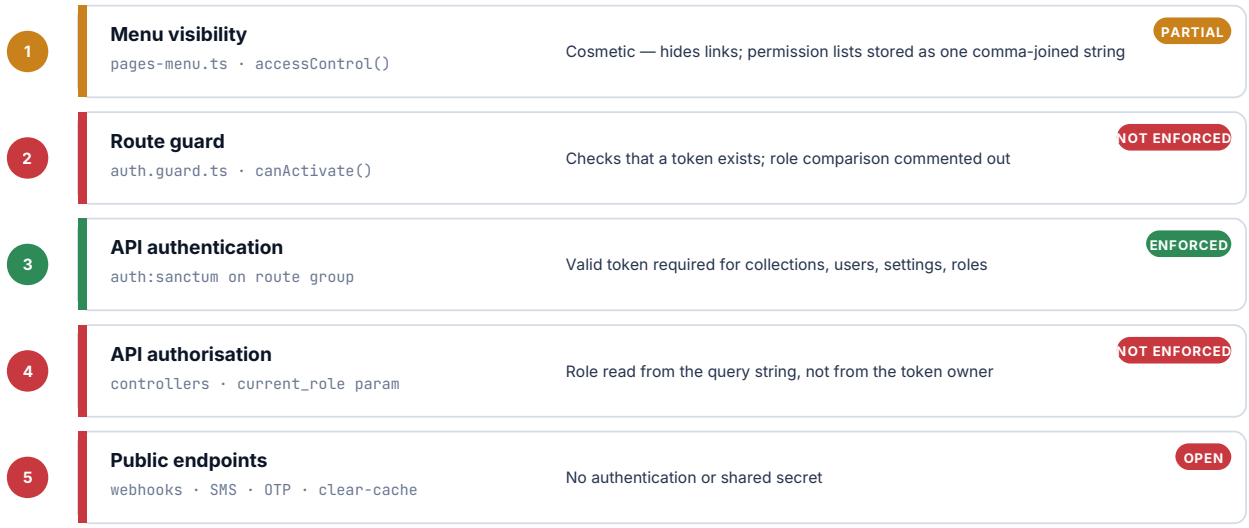


Figure 24 Where access decisions are made today and how strongly each layer enforces them.

11.4 Data scoping

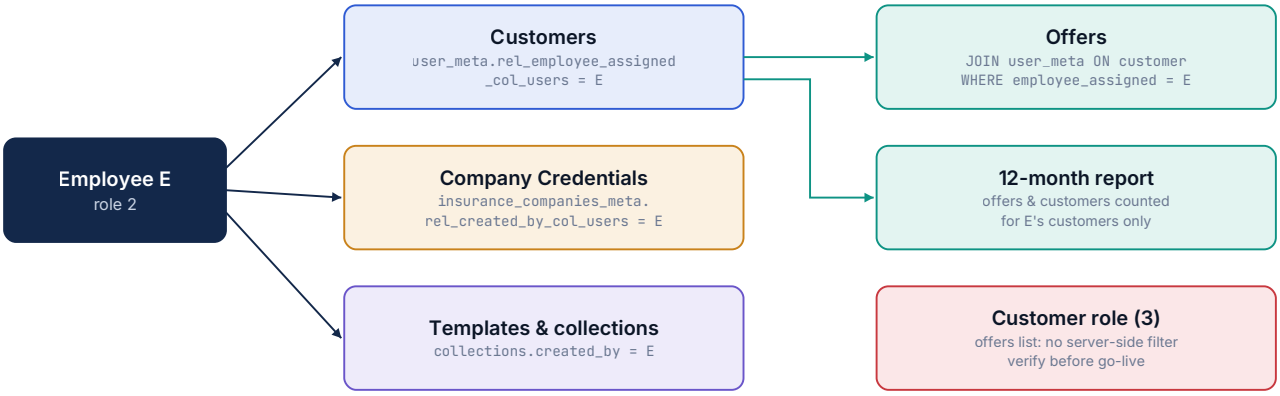


Figure 25 Row-level scoping applied for employees. The customer role receives no server-side filter on the offer list.

DATA	ADMIN	EMPLOYEE	CUSTOMER
Offers	all	offers of assigned customers	all (not filtered)
Customers / users	all users, any role	customers assigned to them	—
Company Credentials	all	rows they created	—
Collections and templates	all	rows they created	—
Relation dropdowns	all options	own rows; assigned customers	—
Reports	whole portfolio	assigned customers only	—

THE KEY WEAKNESS

Every scoping rule above is selected by `current_role` and `current_user_id` from the query string. A customer token with `current_role=admin` receives admin-scoped data, and `/user/register` or `/user/edit/{id}` accept `roles_id` from any token holder. The fix is mechanical: derive the user and role from `$request→user()` in a middleware, ignore the parameters, and move rules into Laravel policies.

PART FOUR

IV

Data

The structure, volume and integrity of the production database, and the reporting built on top of it.

12 Database architecture

13 Reporting

12 Database architecture

The database combines a conventional Laravel schema with a large, application-managed dynamic schema. Of 97 tables, only 14 are described by migrations; the rest were created through the API.

12.1 Schema layers

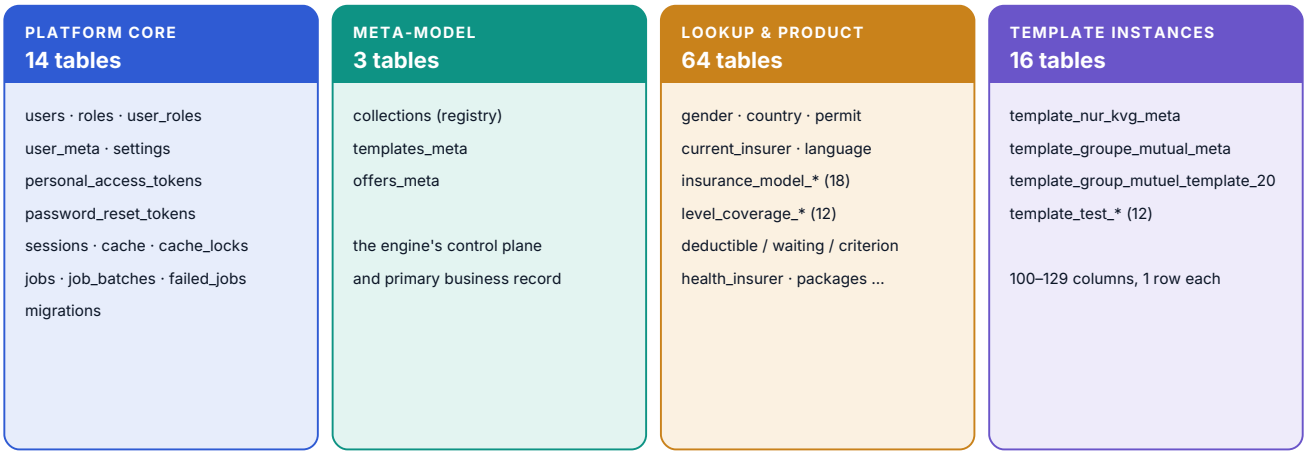


Figure 26 The four layers of the schema. Counts are taken from the export of 22 October 2026.

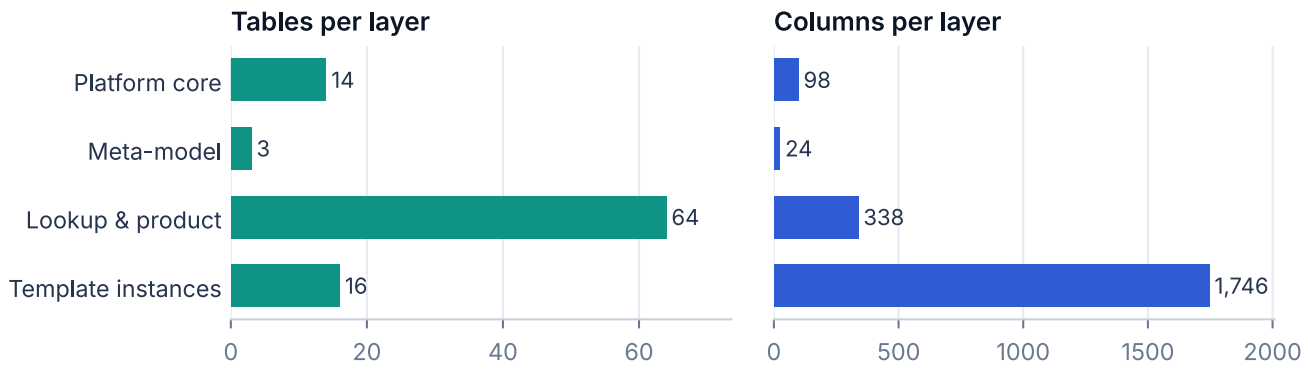


Figure 27 Tables and columns per layer. Template instances are few but account for 79 % of all columns.

12.2 Core entity relationships

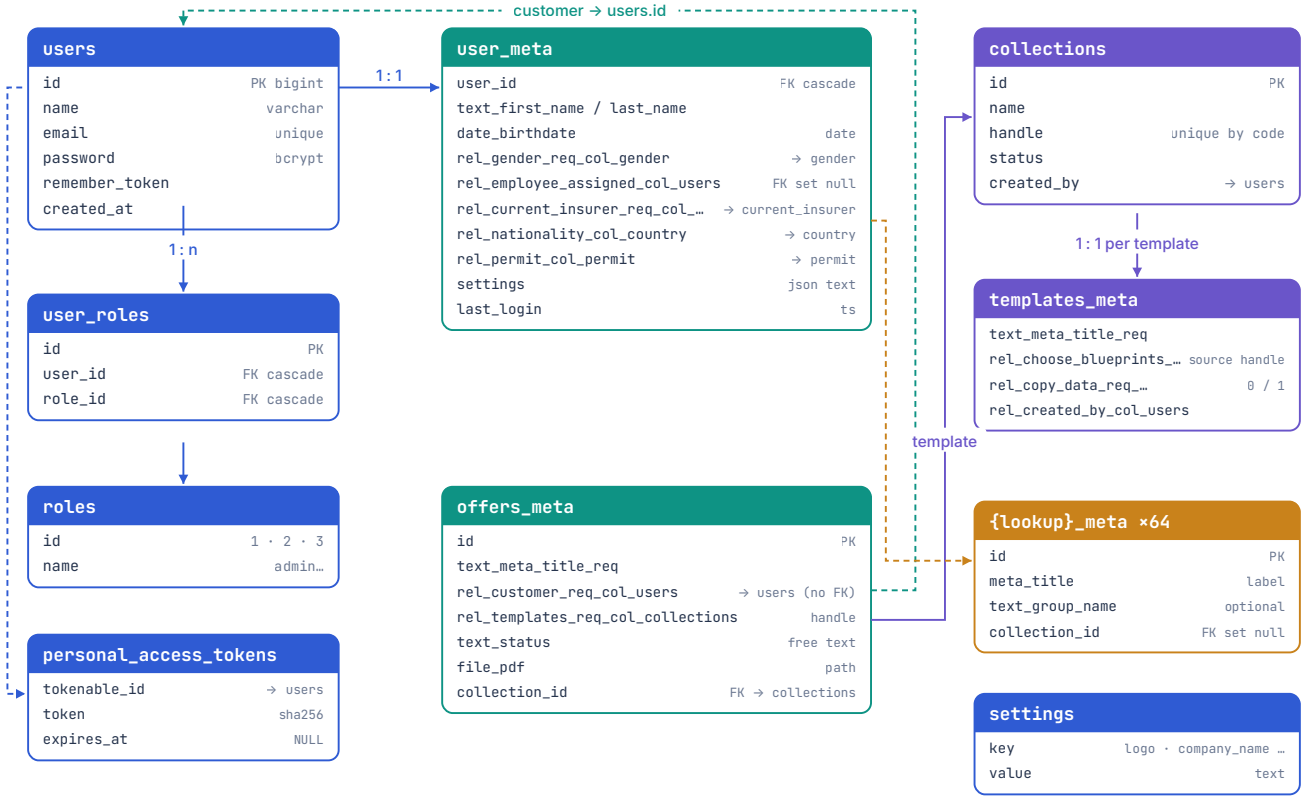


Figure 28 Core entities and relationships. Solid arrows are foreign keys or framework relations; dashed arrows are relations by naming convention only.

Platform tables

TABLE	ROWS	PURPOSE AND NOTABLE COLUMNS
`users`	23	Identity: name, unique e-mail, bcrypt password hash, remember token
`roles` · `user_roles`	3 · 20	Role catalogue and assignment; both FKs cascade on delete
`user_meta`	23	24-column profile: names, address, birth date, phones, doctor, current insurers (KVG and VVG), nationality, permit, assigned employee, settings JSON, last login
`personal_access_tokens`	811	Sanctum tokens (SHA-256 hashes), no expiry
`settings`	3	Key–value: `logo`, `company_name`, `company_email`
`password_reset_tokens` · `sessions`	—	Framework tables (sessions are file-based in practice)
`cache` · `cache_locks`	26 · 0	Database cache store, also used by the login rate limiter
`jobs` · `job_batches` · `failed_jobs`	0	Queue tables; no job classes exist in the code base
`migrations`	20	Includes entries for migration files no longer present (e.g. `create_test_four_meta_table`, several `contracts_meta` variants)

12.3 Volumes

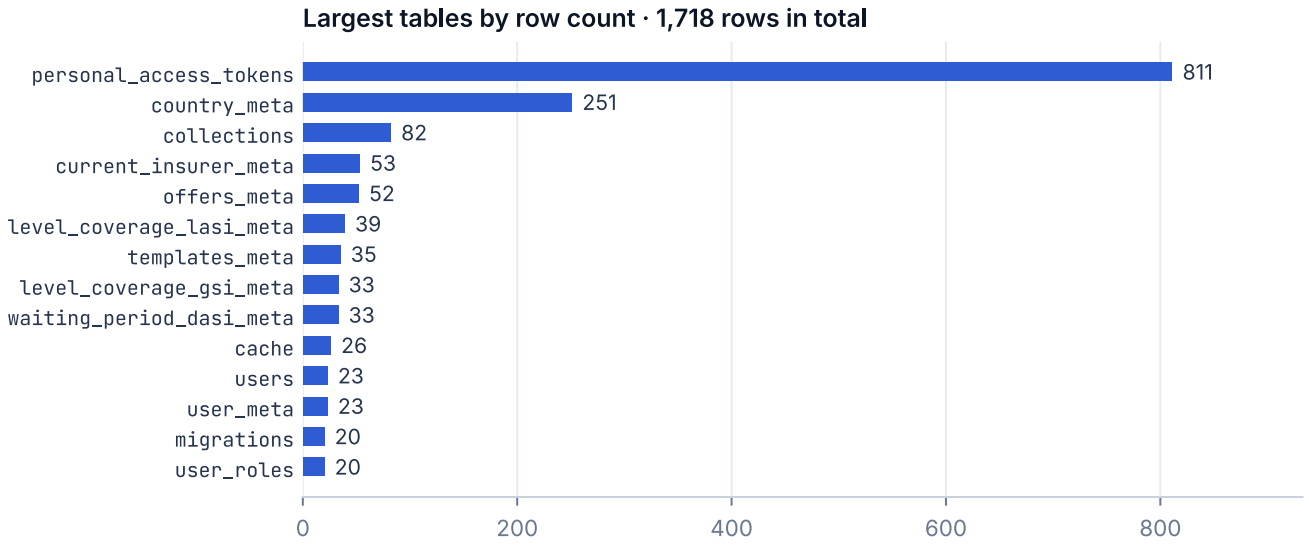


Figure 29 The fourteen largest tables by row count. Apart from tokens, the database is small; its complexity lies in breadth, not volume.

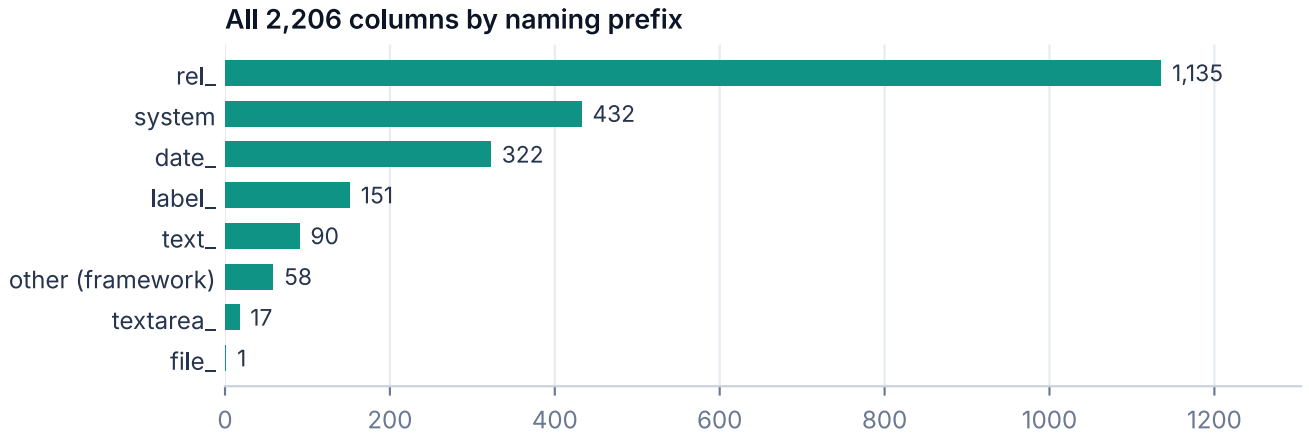


Figure 30 Every column in the database classified by naming prefix. Relations account for more than half.

12.4 Integrity assessment

OBSERVATION	DETAIL AND CONSEQUENCE
Relations without constraints	1,135 `rel_` columns, almost all VARCHAR without foreign keys; deleted lookup rows leave dangling ids that display as raw numbers.
Mixed key types	`rel_customer_req_col_users` is BIGINT in `offers_meta`, while most relations are VARCHAR(255) and `rel_templates_req_col_collections` is MEDIUMTEXT holding a handle.
Dangling references	Two offers reference `template_test_template`, which is not a registered handle.
Accounts without role	3 of 23 users have no `user_roles` row.
Copied templates	`CREATE TABLE … AS SELECT` tables have no primary key.
Migration drift	`settings`, `templates` and `templates_meta` migrations are pending although their tables exist; running `migrate` on the import fails.
No soft deletes, no audit	No table records who changed what; hard deletes everywhere.
Test data in production	12 `template_test_*` tables and registry rows, plus a test account.

12.5 Data protection

`user_meta` holds personal data within the scope of the Swiss Federal Act on Data Protection (nFADP): names, address, date of birth, phone numbers, nationality, residence permit, current insurers and treating doctor. `insurance_companies_meta` holds portal user names and passwords in plain text. Offer PDFs in `public/assets/offers/` are reachable by URL without authentication; file names combine a slug of the uploaded name with a Unix timestamp. Database exports — including the one this document is based on — contain real customer records and must be handled accordingly.

13 Reporting

The dashboard's report is produced by a single endpoint that aggregates the last twelve months of offers and customers, scoped to the advisor's portfolio for non-admins.

13.1 Endpoint

`GET` `/api/user/reports/all` — implemented in `CollectionController::reports()` and consumed by `UsersService` for the ECommerce dashboard's charts panel.

METRIC	DEFINITION
Offers	`offers_meta` rows created in the last 12 months, grouped by `DATE_FORMAT(created_at, '%Y-%m')`. Non-admins: joined to customers whose assigned employee is the caller.
Customers	Users with role 3 created in the last 12 months, grouped by month, same scoping.
Males / Females	Counts of those customers with `rel_gender_req_col_gender` = 2 / 1 (ids of the `gender` lookup).
Customers older than 40	Intended as customers born 40 or more years ago; implemented as a filter on the already-grouped monthly collection, so it does not count what its label says.

13.2 Response shape

```
{ "Customers": [ { "month": "2025-05", "count": 4 }, … ],
  "Customers older than 40 years old": 0,
  "Males": 6, "Females": 5,
  "Offers": [ { "month": "2025-05", "count": 46 }, … ] }
```

Values shown are illustrative of the shape. Because all offers were created in May and June 2025, a report run after June 2026 returns empty offer series.

13.3 Labels and translation

Report labels such as *12 Months Report Summary*, *Offers Created* and *Customers* are part of the translation dictionary and render in the user's language.

PART FIVE

V

Security, Quality and Operations

A risk-ranked security assessment, the state of the code base and its dependencies, the procedures for running the platform, and a sequenced plan to bring it to production grade.

14	Security assessment
15	Code quality and technical debt
16	Operations runbook
17	Remediation roadmap

14 Security assessment

Twenty risks were identified by reading every route, controller, trait, guard and service, the database export and the automation package. Each is rated for impact and likelihood; evidence points to the exact location in the source.

14.1 Method

The assessment is a white-box code review; no tests were executed against the production system. **Impact** considers confidentiality of customer data, integrity of offers and schema, and availability. **Likelihood** considers whether the issue is reachable without authentication, how much knowledge it needs, and whether it is already exposed by normal use. Findings that depend on server configuration (for example PHP execution inside upload folders) are rated on the assumption of Apache defaults and should be confirmed on a copy of the host.

14.2 Risk heat map

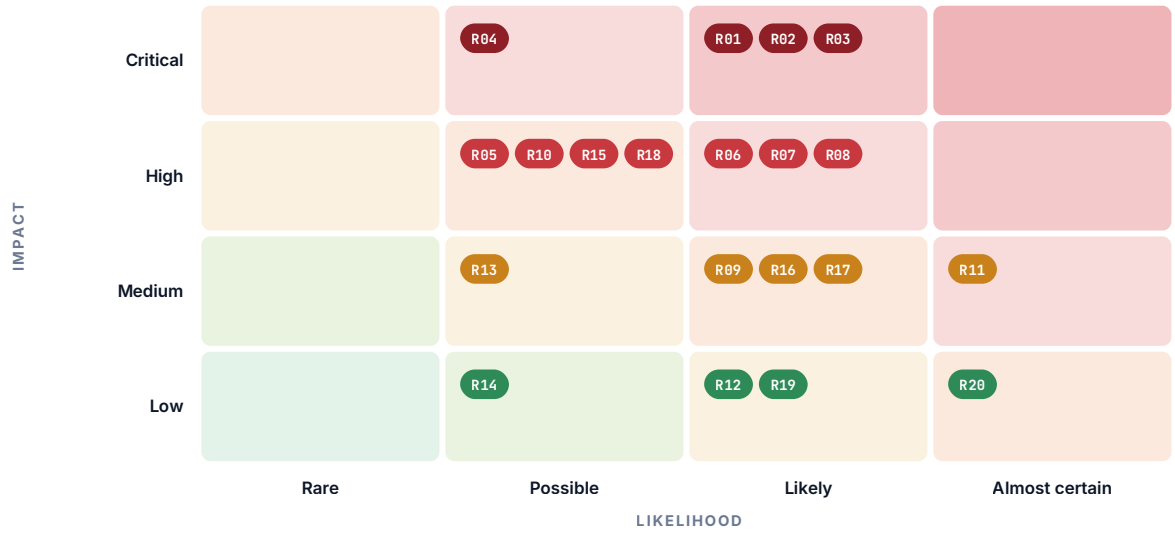


Figure 33 Distribution of the twenty findings by impact and likelihood.

14.3 Summary

ID	FINDING	AREA	IMPACT	LIKELIHOOD
R01	Unauthenticated upload into the public web folder	Integration	CRITICAL	Likely
R02	Any signed-in user can modify any account and its role	Identity	CRITICAL	Likely
R03	Authorisation based on role supplied by the browser	Access control	CRITICAL	Likely
R04	SQL injection in template creation	Data	CRITICAL	Possible
R05	Destructive schema operations open to every role	Data	HIGH	Possible
R06	One-time codes readable by anyone	Integration	HIGH	Likely
R07	Insurer portal passwords stored in plain text	Data	HIGH	Likely
R08	Customers are not scoped to their own offers	Access control	HIGH	Likely

ID	FINDING	AREA	IMPACT	LIKELIHOOD
R09	Offer status can be set by anyone	Integration	MEDIUM	Likely
R10	Flow trigger signature embedded in source code	Secrets	HIGH	Possible
R11	Debug mode and raw exception text in responses	Configuration	MEDIUM	Almost certain
R12	Public cache-clearing endpoint	Configuration	LOW	Likely
R13	Long-lived tokens stored in localStorage	Identity	MEDIUM	Possible
R14	CORS allows every origin	Configuration	LOW	Possible
R15	Archives and old code in web-served folders	Exposure	HIGH	Possible
R16	Failure callbacks target a legacy host	Integration	MEDIUM	Likely
R17	No timeout or recovery in the offer pipeline	Reliability	MEDIUM	Likely
R18	GitHub access token in a supplied repository copy	Secrets	HIGH	Possible
R19	Accounts without a role break the client	Reliability	LOW	Likely
R20	Test data in production	Data	LOW	Almost certain

14.4 Findings

R01 Unauthenticated upload into the public web folder — CRITICAL

CONTEXT Integration · Likely

EVIDENCE routes/api.php:92 · CollectionMetaController::handleWebhook (l. 430–461): rule `'pdf' ⇒ 'required|file|'`; file moved to `public/assets/offers/` with `getClientOriginalExtension()`.

IMPACT Anyone can attach an arbitrary file to any offer id and place it under a web-served path with the extension of their choice. On a default Apache/PHP host a script file would be executable; at minimum, offers can be overwritten with forged documents.

REMEDIATION Authenticate the webhook (HMAC), validate `mimes:pdf` and size, generate the file name server-side, store outside `public/`, serve through an authorised download route.

R02 Any signed-in user can modify any account and its role — CRITICAL

CONTEXT Identity · Likely

EVIDENCE RegisteredUserController::store (l. 39, 81) behind `/api/user/register` and `/api/user/edit/{id}`; rules inferred by `DynamicValidationRules`.

IMPACT A customer token can change another user's e-mail and password (account takeover) or create a new account with `roles_id = 1` (administrator).

REMEDIATION Admin-only routes via policy; explicit validation rules; never accept `roles_id` from non-admins; require the current password for self-service changes.

R03 Authorisation based on role supplied by the browser — CRITICAL

CONTEXT Access control · Likely

EVIDENCE CollectionController::all_collection, show (l. 211, 215), reports (l. 733–774); CollectionMetaController::create_entry / update (l. 139, 309); trait GetRelationalFieldDetail — all read `current_role` / `current_user_id` from the request.

IMPACT Every scoping and ownership rule can be bypassed by editing two query parameters; a customer can read all customers and advisors' insurer credentials.

REMEDIATION Resolve user and role from the token in middleware; remove the parameters from the client; express rules as Laravel policies with tests.

R04 SQL injection in template creation — CRITICAL

CONTEXT	Data · Possible
EVIDENCE	CollectionMetaController::create_template (l. 113–125): `"CREATE TABLE ".$newtable."_meta AS SELECT * FROM $sourcetable"` where `$sourcetable` comes from the request field `rel_choose_blueprints_req_col_collections`.
IMPACT	An authenticated user can inject SQL into a DDL statement executed with the application's database privileges.
REMEDIATION	Look the source handle up in `collections` and use the stored value; quote identifiers; reject anything not matching `^[a-z0-9_]+$`.

R05 Destructive schema operations open to every role — HIGH

CONTEXT	Data · Possible
EVIDENCE	`collections/delete/{handle}` (DROP TABLE), `{handle}/fields/delete` (DROP COLUMN), `add-field`, `fields/edit` — inside `auth:sanctum` only.
IMPACT	Any token holder can drop the offers table or a profile column; there is no audit trail and no soft delete.
REMEDIATION	Admin-only policy, confirmation tokens for destructive calls, audit log, backups before DDL.

R06 One-time codes readable by anyone — HIGH

CONTEXT	Integration · Likely
EVIDENCE	SmsController::getLatestOtp; `GET /api/latest-otp` public; code stored in `public/otp/otp.json`.
IMPACT	Second-factor codes for insurer portal accounts are exposed to the internet, defeating the protection they provide.
REMEDIATION	Store codes privately, keyed by robot run and expiring; require the webhook secret to read; never place them under `public/`.

R07 Insurer portal passwords stored in plain text — HIGH

CONTEXT	Data · Likely
EVIDENCE	`insurance_companies_meta.text_password_req` (VARCHAR); returned by `collections/show/insurance_companies`.
IMPACT	Database exports, backups and API responses disclose credentials for third-party insurer systems.
REMEDIATION	Encrypt at rest (Laravel `encrypted` cast or Crypt), hide from API responses, rotate existing passwords.

R08 Customers are not scoped to their own offers — HIGH

CONTEXT	Access control · Likely
EVIDENCE	CollectionController::show applies filters for employees only; customers' menu includes Offers.
IMPACT	A customer listing offers receives every offer with expanded customer records of other people.
REMEDIATION	Policy: customers see offers where `rel_customer_req_col_users` equals their id; add tests.

R09 Offer status can be set by anyone — MEDIUM

CONTEXT	Integration · Likely
EVIDENCE	`POST /api/webhook/offer-status-update` outside `auth:sanctum`; status written verbatim.
IMPACT	Offers can be marked Ready/Failed or given arbitrary text by third parties.
REMEDIATION	HMAC authentication, enumerated status values.

R10 Flow trigger signature embedded in source code — HIGH

CONTEXT	Secrets · Possible
EVIDENCE	CollectionMetaController::createOffer: full Logic Apps URL including `sig=` parameter.
IMPACT	Anyone with source access can start the production robot for arbitrary offer ids; non-production copies trigger production.
REMEDIATION	Regenerate the trigger, move to `OFFER_FLOW_URL`, use an empty value outside production.

R11 Debug mode and raw exception text in responses — MEDIUM

CONTEXT	Configuration · Almost certain
EVIDENCE	Production `APP_DEBUG=true`; controllers return `$e→getMessage()` including SQL.
IMPACT	Leaks schema, paths and query structure that make other findings easier to exploit.
REMEDIATION	`APP_DEBUG=false`; generic error bodies; log details server-side.

R12 Public cache-clearing endpoint — LOW

CONTEXT	Configuration · Likely
EVIDENCE	routes/api.php:23 `GET /api/clear-cache` runs four Artisan commands without authentication.
IMPACT	Unauthenticated callers can repeatedly flush caches (minor availability impact).
REMEDIATION	Remove; use Artisan during deployment.

R13 Long-lived tokens stored in localStorage — MEDIUM

CONTEXT	Identity · Possible
EVIDENCE	`sanctum.expiration = null`; 811 tokens at export; token kept in `localStorage`.
IMPACT	A token stolen through any script injection remains valid indefinitely.
REMEDIATION	Expiry (e.g. 8 h) with refresh, pruning, consider HttpOnly cookie-based Sanctum SPA auth.

R14 CORS allows every origin — LOW

CONTEXT	Configuration · Possible
EVIDENCE	config/cors.php: `allowed_origins: ['*']`, paths `*`.
IMPACT	Any website can call the API from a browser with a stolen token.
REMEDIATION	Restrict to the production front-end origin.

R15 Archives and old code in web-served folders — HIGH

CONTEXT	Exposure · Possible
EVIDENCE	`site.zip` (306 MB) in the web root; `public/Archive.zip` (89 files); `public/routes-old/`.
IMPACT	Archives may contain source, configuration or data and are downloadable by name.
REMEDIATION	Inspect, remove, and block archive extensions at the web server.

R16 Failure callbacks target a legacy host — MEDIUM

CONTEXT	Integration · Likely
EVIDENCE	Exported flow scopes AgainFailed1 and Timeout1 post to `trustcode.techsolsint.com`.
IMPACT	Failed offers in production never leave *Creating Offer*; the legacy host receives offer ids.
REMEDIATION	Parameterise the callback base URL; verify the deployed flow.

R17 No timeout or recovery in the offer pipeline — MEDIUM

CONTEXT	Reliability · Likely
EVIDENCE	`CURLOPT_TIMEOUT = 0`; no queue job; no sweeper; empty-template guard is silent.
IMPACT	Hung PHP workers and offers stuck indefinitely without anyone being told.
REMEDIATION	Queued job with timeout and retries; scheduled sweeper with alerting; validation errors to the user.

R18 GitHub access token in a supplied repository copy — HIGH

CONTEXT	Secrets · Possible
EVIDENCE	`.git/config` remote URL of a separately supplied back-end archive.
IMPACT	Repository read/write access for whoever holds the archive.
REMEDIATION	Revoke the token; review repository audit log.

R19 Accounts without a role break the client — LOW

CONTEXT	Reliability · Likely
EVIDENCE	3 of 23 users lack `user_roles`; client code dereferences `roles[0]` everywhere.
IMPACT	Affected users see a blank application after login.
REMEDIATION	Enforce a role on creation; guard client code; data fix.

R20 Test data in production — LOW

CONTEXT	Data · Almost certain
EVIDENCE	12 `template_test_*` collections and tables; a test account.
IMPACT	Noise in dropdowns and reports; risk of offers built from test templates (2 exist).
REMEDIATION	Remove after export; keep test data in a staging database.

14.5 Secrets and exposed artefacts

ARTEFACT	LOCATION	ACTION
Power Automate trigger signature	`CollectionMetaController::createOffer`	Regenerate the trigger URL; store as `OFFER_FLOW_URL` in `.env` (the value is intentionally not reproduced here)
GitHub personal access token	`.git/config` of a separately supplied backend copy	Revoke on GitHub; audit repository access log
`site.zip` (306 MB, July 2025)	Production web root	Inspect, move off the web root, delete
`Archive.zip` (89 files, seeders and factories)	`trustcore-backend/public/`	Delete
`routes-old/`	`trustcore-backend/public/`	Delete
`otp/otp.json`	`trustcore-backend/public/`	Move to private storage
Insurer portal passwords	`insurance_companies_meta.text_password_req`	Encrypt with Laravel's encrypter; never return to the browser

15 Code quality and technical debt

The code base delivers a working, flexible product with very little code. The same economy leaves it without tests, with duplicated logic, and with framework versions that are approaching or past end of support.

15.1 Strengths

- **High leverage.** One table component, one form component and one controller pair serve 82 content types.
- **Coherent convention.** The column grammar is applied consistently by back end, front end and field builder.
- **Standard authentication.** Sanctum tokens, bcrypt hashing and a rate-limited login request from Breeze.
- **Operator autonomy.** Staff can add a product dimension or a profile field without a release.

15.2 Debt register

#	ITEM	DETAIL	EFFORT
D1	No automated tests	Only Breeze sample tests; nothing covers collections, offers, scoping or webhooks.	Large
D2	Duplicated controller logic	`validate_collection` and relation expansion loops are copied across controllers; `TemplatesController` (624 lines) is an unrouted parallel implementation.	Medium
D3	Super-global access	`$_GET` / `$_REQUEST` used inside controllers and traits instead of the request object.	Small
D4	Synchronous integration	Offer trigger runs in the web request with no timeout; no queue jobs despite a configured database queue.	Medium
D5	N+1 relation queries	Option lists loaded per cell; no caching.	Medium
D6	Unversioned dynamic schema	Runtime DDL is not recorded anywhere; environments cannot be reproduced from code.	Large
D7	Demo code in the bundle	ngx-admin demo modules (charts, maps, editors, IoT dashboard, e-commerce widgets) remain routed and shipped.	Medium
D8	Front-end debugging output	`console.log` of responses and tokens throughout services and the auth strategy.	Small
D9	Blocking browser dialogs	`window.alert` / `confirm` for all feedback instead of Nebular toasts and dialogs.	Small
D10	Route ordering	Explicit routes declared after `:handle` are unreachable.	Small
D11	Menu built at import time	`MENU_ITEMS` is computed when the module loads, forcing full-page reloads after login.	Small
D12	Branch hygiene	Source of record only on a feature branch; production had only compiled output; leftover `src-old`, `src-bk` folders in the original repository.	Small
D13	Typos in user-facing text	“recieve” in the offer confirmation; inconsistent “Groupe/Group Mutuel” naming.	Small

15.3 Dependency currency

COMPONENT	IN USE	STATUS AND RECOMMENDED DIRECTION
Angular	15.2	Out of long-term support. Upgrade stepwise through 16, 17 and 18 to a supported release; ngcc disappears from 16 onward, removing the Node 18 pin.

COMPONENT	IN USE	STATUS AND RECOMMENDED DIRECTION
Nebular / ngx-admin	11.0.1	Tied to Angular 15; plan the upgrade together with Angular or replace the shell.
`@angular/google-maps`	12.2.13	Three majors behind Angular; remove with the demo maps module.
RxJS (+ rxjs-compat)	6.6	Move to 7 and drop the compatibility package during the Angular upgrade.
Chart.js / Leaflet / TinyMCE / CKEditor	2.7 / 1.2 / 4.5 / 4.7	Demo-only; remove rather than upgrade.
Laravel	11.31	Supported framework line; keep patched via `composer update` within ^11.
PHP	8.2	Supported; plan 8.3 or 8.4 with the next hosting change.
MySQL	8.0.42	MySQL 8.0 reached end of life in April 2026 under Oracle's published schedule; plan a move to an 8.4 LTS server with the host.
Node (build only)	18	End of life; required only because of ngcc.

Support statements reflect vendor schedules as published up to mid-2026; confirm current status before planning.

16 Operations runbook

Procedures for setting up a workstation, releasing changes to the shared host and keeping the running system healthy.

16.1 Local workstation

Prerequisites

TOOL	REQUIREMENT
PHP	8.2+ with `pdo_mysql`, `mbstring`, `openssl`, `fileinfo`, `curl`, `xml`
Composer	2.x
MySQL	8.x (MariaDB 10.6+ untested)
Node	18.x exactly, alongside any newer system Node (nvm or Homebrew `node@18`)

Database

```
mysql -u root -p -e "CREATE DATABASE trustcore CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
mysql -u root -p trustcore < TrustcoreDB.sql          # 97 tables; do NOT run artisan migrate afterwards
```

Back end

```
cd Trustcore-Backend
composer install
cp .env.example .env && php artisan key:generate     # set DB_* and FRONTEND_URL
chmod -R 775 storage bootstrap/cache
php artisan serve --host=127.0.0.1 --port=8000
```

Front end

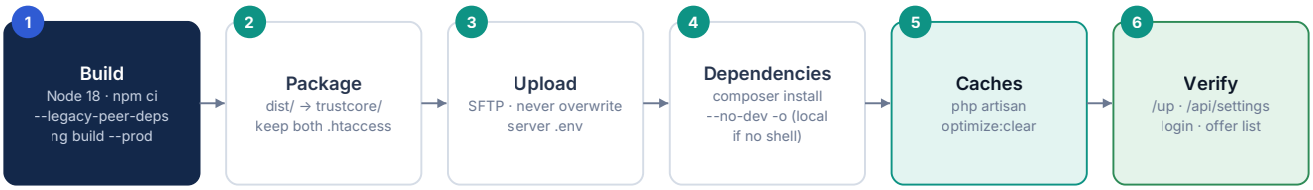
```
cd Trustcore-Frontend
nvm use 18
npm install --legacy-peer-deps
npx ng serve                                          # http://localhost:4200 → API 127.0.0.1:8000
```

Smoke test

1. `php artisan about` reports driver *mysql* and environment *local*.
2. `GET /api/settings` returns company name and logo without a token.
3. `POST /api/login` with an imported account returns 200, a user with roles and a token.
4. The dashboard loads and the menu matches the account's role.

> **NEVER CREATE OFFERS OUTSIDE PRODUCTION**
> Until the trigger URL is configurable, creating an offer on any copy starts the production robot for the production offer that carries the same id.

16.2 Release procedure



Never create offers on non-production copies: the flow trigger is hard-coded to production and the robot will act on the real offer with the same ID.

Figure 31 Release steps for the shared host.

#	STEP	DETAIL
1	Build front end	Node 18, `npm ci --legacy-peer-deps`, `npm run build:prod`; confirm `environment.prod.ts` points at `trustcore.ch`.
2	Package	Contents of `dist/` go to `trustcore/`; keep `deploy/web-root.htaccess` and `deploy/app-folder.htaccess` in place.
3	Upload	SFTP changed files. Never overwrite the server's `.env` (database credentials and `APP_KEY`; a new key invalidates encrypted values).
4	Dependencies	`composer install --no-dev --optimize-autoloader` on the server, or locally and upload `vendor/` when no shell is available.
5	Clear caches	`php artisan optimize:clear`. The public `/api/clear-cache` route exists for shell-less hosts but should be removed (R12).
6	Verify	`/up`, `/api/settings`, login, offer list, one template form.

16.3 Configuration reference

VARIABLE	PRODUCTION VALUE	NOTES
`APP_ENV` / `APP_DEBUG`	production / **false**	Debug is currently `true` in production
`APP_URL` / `FRONTEND_URL`	https://trustcore.ch…	Used for generated links (password reset)
`DB_*`	host credentials	Kept in the password manager
`SESSION_DRIVER` / `CACHE_STORE` / `QUEUE_CONNECTION`	file / database / database	A queue worker is needed once offer triggering is queued
`MAIL_*`	real SMTP	`log` in the template: password-reset e-mails are not delivered unless configured
`OFFER_FLOW_URL` (proposed)	signed trigger URL	Empty outside production disables triggering
`WEBHOOK_SECRET` (proposed)	random 32+ bytes	Shared with the flow for callback authentication

16.4 Routine operations

TASK	FREQUENCY	PROCEDURE
Database backup	Daily	Hoststar backup plus an encrypted `mysqldump` off-site; the schema changes at runtime, so backups are the only record of structure.
Offer PDF backup	Daily	`public/assets/offers/` (113 files at export)
Stuck offers	Daily	List offers in *Creating Offer* older than one hour; check the flow run history.

TASK	FREQUENCY	PROCEDURE
Flow health	Weekly	Power Automate run history for failures; robot machine availability.
Token hygiene	Monthly	Prune tokens unused for 90 days (`last_used_at`).
Log review	Weekly	`storage/logs/laravel.log` ; note it contains full inbound SMS payloads.
Dependency updates	Monthly	`composer update` within ^11; `npm audit` .

16.5 Troubleshooting

SYMPTOM	LIKELY CAUSE AND RESOLUTION
Login works but dashboard is blank	Account has no role; `roles[0]` is undefined. Assign a role in `user_roles` .
`npm install` fails with ERESOLVE	Missing `--legacy-peer-deps` .
Install fails in `ngcc`	Node newer than 18.
Offer stays “Creating Offer”	Empty template table, trigger failure, robot offline, or failure callback sent to the legacy host.
Offer “Failed”	Robot failed twice; check the desktop flow log and the customer and template data it reads.
Menu items missing after login	Role permissions as designed; or stale localStorage — sign out and in.
500 with SQL text in the response	A submitted key does not exist as a column; check the form against the blueprint.
`artisan migrate` fails on import	Expected: three migrations are pending for existing tables. Do not migrate imported databases.

17 Remediation roadmap

Fixes are sequenced so that exposure is contained first, authorisation is moved to the server second, and operability and modernisation follow on a stable base.



Figure 32 Sixteen-week remediation plan in four phases. Critical findings are closed by the end of Phase 1 (week 6).

17.1 Phase 0 — Contain (weeks 1–2)

- Require a shared secret (HMAC header) on both webhooks and the SMS endpoint; remove `/latest-otp` from the public API or protect it with the same secret; delete `/clear-cache`. R01, R06, R09, R12
- Validate the webhook upload as `mimes:pdf|max:…`, store under `storage/` with a generated name and serve through an authenticated download route. R01
- Set `APP_DEBUG=false`; return generic error messages. R11
- Remove `site.zip`, `Archive.zip`, `routes-old/` and test data; revoke the leaked GitHub token; regenerate the flow trigger URL. R10, R15, R18, R20
- Restrict `/user/register`, `/user/edit` and all schema endpoints to administrators using the token owner's role. R02, R05

17.2 Phase 1 — Authorise (weeks 3–6)

- Middleware that resolves user and role from `$request->user()`; delete every use of `current_role` / `current_user_id`. R03
- Laravel policies per collection type; customer offer scoping; employee rules preserved. R08
- Whitelist handles against the registry and validate column names against the grammar before any DDL or dynamic table access. R04
- Encrypt insurer credentials at rest; expose them only to the robot. R07

- Finish or remove the Angular route guard; fix the permission lists; add a role fallback. R19

17.3 Phase 2 — Operate (weeks 7–9)

- Queue the offer trigger as a job with timeout and retries; scheduled sweeper for stuck offers; snapshot the template row into the offer. R17
- Trigger URL and callback base URL per environment; a staging flow. R10, R16
- Token expiry and pruning; move the front-end token out of localStorage where feasible; restrict CORS to the front-end origin. R13, R14
- Backups, uptime monitoring, an audit log of entry and schema changes.

17.4 Phase 3 — Modernise (weeks 10–16)

- Angular upgrade to a supported release; remove demo modules and unused dependencies.
- Feature tests for collections, entries, offers, webhooks and scoping; CI pipeline running lint, tests and the production build.
- Record runtime DDL as generated migrations so every environment can be rebuilt from code.

17.5 Definition of done

AREA	ACCEPTANCE CRITERIA
Access control	Automated tests prove that each role can reach exactly the rows in section 11.4 regardless of request parameters.
Integration	Unsigned webhook calls are rejected; uploads other than PDF are rejected; every offer reaches a terminal state within a defined time.
Schema safety	No SQL statement contains unvalidated input; DDL endpoints are admin-only and logged.
Operations	Restore from backup rehearsed; staging environment with its own flow; alerts on failed and stuck offers.

APPENDICES

A

Reference Data

Complete catalogues generated from the database export and the source tree: every table, every template field, every lookup and registered collection, the source inventory and a glossary.

A	Table catalogue
B	Template field dictionary
C	Lookup collections
D	Collection registry
E	Source inventory
F	Glossary

A Table catalogue

All 97 tables in the export of 22 October 2026 with their layer, column count, row count and number of declared foreign keys.

TABLE	LAYER	COLUMNS	ROWS	FKS
accident_coverage_dasi_meta	Lookup	6	4	1
accident_coverage_meta	Lookup	5	2	1
assets_meta	Lookup	4	0	1
boolean_meta	Lookup	5	2	1
cache	Platform	3	26	
cache_locks	Platform	3	0	
collections	Meta-model	7	82	
communication_meta	Lookup	4	0	1
contracts_meta	Lookup	10	0	4
country_meta	Lookup	5	251	1
coverage_level_meta	Lookup	5	2	1
criterion_cpsi_meta	Lookup	5	2	1
criterion_lsi_meta	Lookup	5	2	1
current_insurer_meta	Lookup	5	53	1
deductible_coverage_hsi_meta	Lookup	6	6	1
deductible_coverage_meta	Lookup	6	12	1
deductible_coverage_shi_meta	Lookup	6	4	1
deductible_meta	Lookup	5	6	1
failed_jobs	Platform	7	0	
franchise_data_meta	Lookup	5	10	1
gender_meta	Lookup	5	2	1
health_insurance_meta	Lookup	5	1	1
health_insurer_dasi_meta	Lookup	6	3	1
health_insurer_meta	Lookup	5	3	1
insurance_companies_meta	Lookup	9	3	1
insurance_model_asi_meta	Lookup	5	1	1
insurance_model_cpsi_meta	Lookup	5	2	1
insurance_model_dasi_meta	Lookup	5	2	1
insurance_model_dsi_meta	Lookup	5	1	1
insurance_model_gsi_meta	Lookup	5	7	1
insurance_model_hadsi_meta	Lookup	5	1	1
insurance_model_hasi_meta	Lookup	5	1	1
insurance_model_hlsi_meta	Lookup	5	1	1
insurance_model_hsi_meta	Lookup	5	3	1
insurance_model_hssi_meta	Lookup	5	1	1

TABLE	LAYER	COLUMNS	ROWS	FKS
insurance_model_isi_meta	Lookup	5	1	1
insurance_model_lasi_meta	Lookup	5	2	1
insurance_model_liasi_meta	Lookup	5	1	1
insurance_model_lsi_meta	Lookup	5	1	1
insurance_model_lsisi_meta	Lookup	5	1	1
insurance_model_lssi_meta	Lookup	5	1	1
insurance_model_mtsi_meta	Lookup	5	1	1
insurance_model_shi_meta	Lookup	5	3	1
insurance_type_meta	Lookup	5	5	1
invoicing_meta	Lookup	5	4	1
job_batches	Platform	10	0	
jobs	Platform	7	0	
land_bank_meta	Lookup	5	0	1
language_meta	Lookup	5	1	1
level_coverage_dasi_meta	Lookup	6	4	1
level_coverage_dsi_meta	Lookup	5	3	1
level_coverage_gsi_meta	Lookup	6	33	1
level_coverage_hlsi_meta	Lookup	5	11	1
level_coverage_hsi_meta	Lookup	6	8	1
level_coverage_isi_meta	Lookup	5	2	1
level_coverage_lasi_meta	Lookup	6	39	1
level_coverage_liasi_meta	Lookup	5	5	1
level_coverage_lsi_meta	Lookup	5	3	1
level_coverage_lsisi_meta	Lookup	5	1	1
level_coverage_mtsi_meta	Lookup	5	2	1
level_coverage_shi_meta	Lookup	6	6	1
migrations	Platform	3	20	
niveau_package_meta	Lookup	5	5	1
occupational_status_meta	Lookup	5	6	1
offers_meta	Meta-model	9	52	1
packages_lca_meta	Lookup	5	2	1
password_reset_tokens	Platform	3	3	
payment_method_meta	Lookup	5	4	1
permit_meta	Lookup	5	13	1
personal_access_tokens	Platform	10	811	
relationship_meta	Lookup	5	1	1
roles	Platform	4	3	
sessions	Platform	6	0	
settings	Platform	5	3	
supplementary_insurance_vvg_meta	Lookup	5	10	1

TABLE	LAYER	COLUMNS	ROWS	FKS
`template_group_mutuel_template_20_meta`	Template	129	1	
`template_groupe_mutual_meta`	Template	129	1	1
`template_groupe_mutuel_template_meta`	Template	100	1	
`template_nur_kvg_meta`	Template	101	1	
`template_test52526373_meta`	Template	100	1	
`template_test5_meta`	Template	100	1	
`template_test765343678697867565_meta`	Template	100	1	
`template_test_10_meta`	Template	129	1	
`template_test_11_meta`	Template	129	1	
`template_test_2_meta`	Template	100	1	
`template_test_6_meta`	Template	100	1	
`template_test_7_meta`	Template	100	1	
`template_test_89_meta`	Template	100	1	
`template_test_8_meta`	Template	100	1	
`template_test_9_meta`	Template	129	1	
`template_test_template1_meta`	Template	100	1	
`templates_meta`	Meta-model	8	35	1
`user_meta`	Platform	24	23	2
`user_roles`	Platform	5	20	2
`users`	Platform	8	23	
`waiting_period_dasi_meta`	Lookup	6	33	1
`waiting_period_isi_meta`	Lookup	5	1	1

B Template field dictionary

The 129 columns of `template_groupe_mutual_meta` in physical order, classified by the column grammar.

#	COLUMN	KIND	TARGET	SQL TYPE	REQ
1	`id`	system		`bigint`	
2	`meta_title`	system		`varchar(100)`	
3	`rel_payment_method_col_payment_method`	relation	`payment_method`	`bigint`	
4	`rel_invoicing_col_invoicing`	relation	`invoicing`	`bigint`	
5	`rel_occupational_status_req_col_occupational_status`	relation	`occupational_status`	`varchar(100)`	●
6	`text_other_occupational_data`	text		`varchar(100)`	
7	`rel_franchise_data_col_franchise_data`	relation	`franchise_data`	`bigint`	
8	`collection_id`	system		`bigint`	
9	`created_at`	system		`timestamp`	
10	`updated_at`	system		`timestamp`	
11	`label_LAMal/KVG_basic_insurance`	section label		`varchar(1)`	
12	`date_come_into_effect_req`	date		`varchar(100)`	●
13	`rel_basic_insurance_kvg_col_insurance_type`	relation	`insurance_type`	`bigint`	
14	`rel_health_insurance_col_health_insurer`	relation	`health_insurer`	`bigint`	
15	`rel_foreign_insurance_country_col_country`	relation	`country`	`bigint`	
16	`label_LCA/VVG_supplemental_insurance`	section label		`varchar(1)`	
17	`date_entry_into_force_vvg`	date		`varchar(100)`	
18	`rel_accidental_coverage_col_boolean`	product switch	`boolean`	`varchar(100)`	
19	`label_Global_range`	section label		`varchar(1)`	
20	`rel_si_with_global_col_boolean`	product switch	`boolean`	`varchar(100)`	
21	`label_Global`	section label		`varchar(1)`	
22	`rel_insurance_model_gsi_col_insurance_model_gsi`	relation	`insurance_model_gsi`	`varchar(100)`	
23	`rel_level_coverage_gsi_col_level_coverage_gsi`	relation	`level_coverage_gsi`	`varchar(100)`	
24	`rel_deductible_coverage_gsi_col_deductible_coverage`	relation	`deductible_coverage`	`varchar(100)`	
25	`date_entry_into_force_gsi`	date		`date`	
26	`rel_facilitated_transfer_gsi_col_boolean`	product switch	`boolean`	`varchar(100)`	
27	`label_Hospitalisation_&_hospitalisation_lump-sum`	section label		`varchar(1)`	
28	`rel_hospitalisation_lump_sum_col_boolean`	product switch	`boolean`	`varchar(100)`	
29	`label_Hospitalisation_lump-sum`	section label		`varchar(1)`	
30	`rel_insurance_model_hlsi_col_insurance_model_hlsi`	relation	`insurance_model_hlsi`	`varchar(100)`	
31	`rel_level_coverage_hlsi_col_level_coverage_hlsi`	relation	`level_coverage_hlsi`	`varchar(100)`	
32	`date_entry_into_force_hlsi`	date		`date`	
33	`label_Hospitalisation`	section label		`varchar(1)`	
34	`rel_hospitalisation_col_boolean`	product switch	`boolean`	`varchar(100)`	

#	COLUMN	KIND	TARGET	SQL TYPE	REQ
35	`rel_insurance_model_hsi_col_insurance_model_hsi`	relation	`insurance_model_hsi`	`varchar(100)`	
36	`rel_level_coverage_hsi_col_level_coverage_hsi`	relation	`level_coverage_hsi`	`varchar(100)`	
37	`rel_deductible_coverage_hsi_col_deductible_coverage_hsi`	relation	`deductible_coverage_hsi`	`varchar(100)`	
38	`date_entry_into_force_hsi`	date		`date`	
39	`label_Travel_insurance`	section label		`varchar(1)`	
40	`rel_mundo_tsi_col_boolean`	product switch	`boolean`	`varchar(100)`	
41	`label_Mundo`	section label		`varchar(1)`	
42	`rel_insurance_model_mundo_col_insurance_model_mtsi`	relation	`insurance_model_mtsi`	`varchar(100)`	
43	`rel_level_coverage_mundo_col_level_coverage_mtsi`	relation	`level_coverage_mtsi`	`varchar(100)`	
44	`date_entry_into_force_mundo`	date		`date`	
45	`label_Dental_insurance`	section label		`varchar(1)`	
46	`rel_dentaire_plus_col_boolean`	product switch	`boolean`	`varchar(100)`	
47	`label_Dentaire_plus`	section label		`varchar(1)`	
48	`rel_insurance_model_dsi_col_insurance_model_dsi`	relation	`insurance_model_dsi`	`varchar(100)`	
49	`rel_level_coverage_dsi_col_level_coverage_dsi`	relation	`level_coverage_dsi`	`varchar(100)`	
50	`date_entry_into_force_dsi`	date		`date`	
51	`label_Supplemental_health_insurance`	section label		`varchar(1)`	
52	`rel_alterna_col_boolean`	product switch	`boolean`	`varchar(100)`	
53	`label_Alterna`	section label		`varchar(1)`	
54	`rel_insurance_model_ahsi_col_insurance_model_hasi`	relation	`insurance_model_hasi`	`varchar(100)`	
55	`rel_level_coverage_ahsi_col_level_coverage_mtsi`	relation	`level_coverage_mtsi`	`varchar(100)`	
56	`date_entry_into_force_ahsi`	date		`date`	
57	`label_Special_care`	section label		`varchar(1)`	
58	`rel_special_care_col_boolean`	product switch	`boolean`	`varchar(100)`	
59	`rel_insurance_model_scsi_col_insurance_model_hssi`	relation	`insurance_model_hssi`	`varchar(100)`	
60	`date_entry_into_force_scsi`	date		`date`	
61	`label_supplemental_health_insurance_s`	section label		`varchar(1)`	
62	`rel_supplemental_health_insurance_col_boolean`	product switch	`boolean`	`varchar(100)`	
63	`rel_insurance_model_shi_col_insurance_model_shi`	relation	`insurance_model_shi`	`varchar(100)`	
64	`rel_level_coverage_shi_col_level_coverage_shi`	relation	`level_coverage_shi`	`varchar(100)`	
65	`rel_deductible_coverage_shi_col_deductible_coverage_shi`	relation	`deductible_coverage_shi`	`varchar(100)`	
66	`date_entry_into_force_shi`	date		`date`	
67	`rel_facilitated_transfer_shi_col_boolean`	product switch	`boolean`	`varchar(100)`	
68	`label_Legal_protection`	section label		`varchar(1)`	
69	`rel_legis_col_boolean`	product switch	`boolean`	`varchar(100)`	
70	`label_Legis`	section label		`varchar(1)`	
71	`rel_insurance_model_lsi_col_insurance_model_lsi`	relation	`insurance_model_lsi`	`varchar(100)`	
72	`rel_level_coverage_lsi_col_level_coverage_lsi`	relation	`level_coverage_lsi`	`varchar(100)`	
73	`rel_criterion_lsi_col_criterion_lsi`	relation	`criterion_lsi`	`varchar(100)`	

#	COLUMN	KIND	TARGET	SQL TYPE	REQ
74	`date_entry_into_force_lsi`	date		`date`	
75	`label_Legis_sana`	section label		`varchar(1)`	
76	`rel_legis_sana_col_boolean`	product switch	`boolean`	`varchar(100)`	
77	`rel_insurance_model_lssi_col_insurance_model_lssi`	relation	`insurance_model_lssi`	`varchar(100)`	
78	`date_entry_into_force_lssi`	date		`date`	
79	`label_Daily_allowance`	section label		`varchar(1)`	
80	`rel_daily_allowance_col_boolean`	product switch	`boolean`	`varchar(100)`	
81	`label_Daily allowance`	section label		`varchar(1)`	
82	`rel_insurance_model_dasi_col_insurance_model_dasi`	relation	`insurance_model_dasi`	`varchar(100)`	
83	`rel_health_insurer_dasi_col_health_insurer_dasi`	relation	`health_insurer_dasi`	`varchar(100)`	
84	`rel_level_coverage_dasi_col_level_coverage_dasi`	relation	`level_coverage_dasi`	`varchar(100)`	
85	`rel_accident_coverage_dasi_col_accident_coverage_dasi`	relation	`accident_coverage_dasi`	`varchar(100)`	
86	`rel_waiting_period_dasi_col_waiting_period_dasi`	relation	`waiting_period_dasi`	`varchar(100)`	
87	`text_daily_amount_dasi`	text		`varchar(100)`	
88	`date_entry_into_force_dasi`	date		`varchar(100)`	
89	`label_Incapacity_for_housework`	section label		`varchar(1)`	
90	`rel_incapacity_for_housework_col_boolean`	product switch	`boolean`	`varchar(100)`	
91	`rel_insurance_model_isi_col_insurance_model_isi`	relation	`insurance_model_isi`	`varchar(255)`	
92	`rel_level_coverage_isi_col_level_coverage_isi`	relation	`level_coverage_isi`	`varchar(255)`	
93	`rel_waiting_period_isi_col_waiting_period_isi`	relation	`waiting_period_isi`	`varchar(255)`	
94	`date_entry_into_force_isi`	date		`date`	
95	`label_Hospitalisation_allowance`	section label		`varchar(1)`	
96	`rel_hospitalisation_allowance_col_boolean`	product switch	`boolean`	`varchar(255)`	
97	`rel_insurance_model_hadsi_col_insurance_model_hadsi`	relation	`insurance_model_hadsi`	`varchar(255)`	
98	`text_daily_amount_hadsi`	text		`varchar(255)`	
99	`date_entry_into_force_hadsi`	date		`date`	
100	`label_Accident_insurance`	section label		`varchar(1)`	
101	`rel_accident_insurance_col_boolean`	product switch	`boolean`	`varchar(255)`	
102	`label_Accident insurance`	section label		`varchar(1)`	
103	`rel_insurance_model_asi_col_insurance_model_asi`	relation	`insurance_model_asi`	`varchar(255)`	
104	`date_entry_into_force_asi`	date		`date`	
105	`label_Insurance for risks and disputes on the Internet`	section label		`varchar(1)`	
106	`rel_cyberprotect_col_boolean`	product switch	`boolean`	`varchar(255)`	
107	`label_CyberProtect`	section label		`varchar(1)`	
108	`rel_insurance_model_cpsi_col_insurance_model_cpsi`	relation	`insurance_model_cpsi`	`varchar(255)`	
109	`rel_criterion_cpsi_col_criterion_cpsi`	relation	`criterion_cpsi`	`varchar(255)`	
110	`date_entry_into_force_cpsi`	date		`date`	
111	`label_Lump-sum in the event of disability or death.`	section label		`varchar(1)`	

#	COLUMN	KIND	TARGET	SQL TYPE	REQ
112	`rel_accident_col_boolean`	product switch	`boolean`	`varchar(255)`	
113	`label_Accident`	section label		`varchar(1)`	
114	`rel_insurance_model_lasi_col_insurance_model_lasi`	relation	`insurance_model_lasi`	`varchar(255)`	
115	`rel_level_coverage_lasi_col_level_coverage_lasi`	relation	`level_coverage_lasi`	`varchar(255)`	
116	`date_entry_into_force_lasi`	date		`varchar(100)`	
117	`label_Illness/Accident`	section label		`varchar(1)`	
118	`rel_IllnessAccident_col_boolean`	product switch	`boolean`	`varchar(100)`	
119	`rel_insurance_model_liasi_col_insurance_model_liasi`	relation	`insurance_model_liasi`	`varchar(255)`	
120	`rel_level_coverage_liasi_col_level_coverage_liasi`	relation	`level_coverage_liasi`	`varchar(255)`	
121	`rel_accident_coverage_liasi_col_accident_coverage`	relation	`accident_coverage`	`varchar(255)`	
122	`date_entry_into_force_liasi`	date		`date`	
123	`label_Accident or sudden illness`	section label		`varchar(1)`	
124	`rel_Accident_or_sudden_illness_col_boolean`	product switch	`boolean`	`varchar(100)`	
125	`rel_insurance_model_lsisi_col_insurance_model_lsisi`	relation	`insurance_model_lsisi`	`varchar(255)`	
126	`rel_level_coverage_lsisi_col_level_coverage_lsisi`	relation	`level_coverage_lsisi`	`varchar(255)`	
127	`date_entry_into_force_lsisi`	date		`date`	
128	`textarea_template_description`	long text		`longtext`	
129	`rel_company_col_insurance_companies`	relation	`insurance_companies`	`varchar(255)`	

C Lookup collections

The 64 lookup and product tables. The standard shape is `id, meta_title, collection_id, created_at, updated_at`; additional columns are listed.

COLLECTION HANDLE	ROWS	COLUMNS BEYOND THE STANDARD LOOKUP SHAPE
accident_coverage_dasi	4	text_group_name
accident_coverage	2	—
assets	0	—
boolean	2	—
communication	0	—
contracts	0	text_title, date_contract_date, textarea_content, rel_created_by_col_users, rel_signed_by_col_users, rel_insurance_type_col_insurance_type
country	251	—
coverage_level	2	—
criterion_cpsi	2	—
criterion_lsi	2	—
current_insurer	53	—
deductible_coverage_hsi	6	text_group_name
deductible_coverage	12	text_group_name
deductible_coverage_shi	4	text_group_name
deductible	6	—
franchise_data	10	—
gender	2	—
health_insurance	1	—
health_insurer_dasi	3	text_group_name
health_insurer	3	—
insurance_companies	3	text_company_name, text_user_name_req, text_password_req, text_mobile, rel_created_by_col_users
insurance_model_asi	1	—
insurance_model_cpsi	2	—
insurance_model_dasi	2	—
insurance_model_dsi	1	—
insurance_model_gsi	7	—
insurance_model_hadsi	1	—
insurance_model_hasi	1	—
insurance_model_hlsi	1	—
insurance_model_hsi	3	—
insurance_model_hssi	1	—
insurance_model_isi	1	—
insurance_model_lasi	2	—

COLLECTION HANDLE	ROWS	COLUMNS BEYOND THE STANDARD LOOKUP SHAPE
insurance_model_liasi	1	—
insurance_model_lsi	1	—
insurance_model_lsisi	1	—
insurance_model_lssi	1	—
insurance_model_mtsi	1	—
insurance_model_shi	3	—
insurance_type	5	text_title
invoicing	4	—
land_bank	0	—
language	1	—
level_coverage_dasi	4	text_group_name
level_coverage_dsi	3	—
level_coverage_gsi	33	text_group_name
level_coverage_hlsi	11	—
level_coverage_hsi	8	text_group_name
level_coverage_isi	2	—
level_coverage_lasi	39	text_group_name
level_coverage_liasi	5	—
level_coverage_lsi	3	—
level_coverage_lsisi	1	—
level_coverage_mtsi	2	—
level_coverage_shi	6	text_group_name
niveau_package	5	—
occupational_status	6	—
packages_lca	2	—
payment_method	4	—
permit	13	—
relationship	1	—
supplementary_insurance_vvg	10	—
waiting_period_dasi	33	text_group_name
waiting_period_isi	1	—

D Collection registry

Every row of the `collections` registry and whether its physical table exists.

ID	NAME	HANDLE	TABLE
11	Contracts	`contracts`	✓
12	Insurance Type	`insurance_type`	✓
13	Insurance Companies	`insurance_companies`	✓
14	Communication	`communication`	✓
16	Assets	`assets`	✓
17	Offers	`offers`	✓
20	Gender	`gender`	✓
22	Health insurance	`health_insurance`	✓
23	Current insurer	`current_insurer`	✓
24	Coverage Level	`coverage_level`	✓
25	Supplementary Insurance Vvg	`supplementary_insurance_vvg`	✓
26	Language	`language`	✓
27	Relationship	`relationship`	✓
28	Country	`country`	✓
29	Payment Method	`payment_method`	✓
30	Land Bank	`land_bank`	✓
31	Invoicing	`invoicing`	✓
32	Boolean	`boolean`	✓
33	Groupe Mutuel Template 1	`template_groupe_mutual`	✓
36	Health insurer	`health_insurer`	✓
37	Deductible	`deductible`	✓
40	Franchise Data	`franchise_data`	✓
41	Occupational Status	`occupational_status`	✓
42	Packages LCA	`packages_lca`	✓
46	niveau Package	`niveau_package`	✓
47	Templates	`templates`	✓
52	Insurance Model GSI	`insurance_model_gsi`	✓
53	Level coverage GSI	`level_coverage_gsi`	✓
54	Deductible coverage GSI	`deductible_coverage`	✓
55	Insurance model HLSI	`insurance_model_hlsi`	✓
56	Level coverage HLSI	`level_coverage_hlsi`	✓
57	Insurance model HSI	`insurance_model_hsi`	✓
58	Level Coverage HSI	`level_coverage_hsi`	✓
59	Deductible coverage HSI	`deductible_coverage_hsi`	✓
60	Insurance model MTSI	`insurance_model_mtsi`	✓
61	Level coverage MTSI	`level_coverage_mtsi`	✓

ID	NAME	HANDLE	TABLE
62	Insurance Model DSI	`insurance_model_dsi`	✓
63	Level coverage DSI	`level_coverage_dsi`	✓
64	Insurance model HASI	`insurance_model_hasi`	✓
65	Insurance model HSSI	`insurance_model_hssi`	✓
66	Insurance model SHI	`insurance_model_shi`	✓
67	Level coverage SHI	`level_coverage_shi`	✓
68	Deductible coverage SHI	`deductible_coverage_shi`	✓
69	Insurance model LSI	`insurance_model_lsi`	✓
70	Level coverage LSI	`level_coverage_lsi`	✓
71	Criterion LSI	`criterion_lsi`	✓
72	Insurance Model LSSI	`insurance_model_lssi`	✓
73	Insurance model DASI	`insurance_model_dasi`	✓
74	Health insurer DASI	`health_insurer_dasi`	✓
75	Level coverage DASI	`level_coverage_dasi`	✓
76	Accident coverage DASI	`accident_coverage_dasi`	✓
77	Waiting period DASI	`waiting_period_dasi`	✓
78	Insurance model ISI	`insurance_model_isi`	✓
79	Level coverage ISI	`level_coverage_isi`	✓
80	Waiting period ISI	`waiting_period_isi`	✓
81	Insurance model HADSI	`insurance_model_hadsi`	✓
82	Insurance model ASI	`insurance_model_asi`	✓
83	Insurance model CPSI	`insurance_model_cpsi`	✓
84	Criterion CPSI	`criterion_cpsi`	✓
85	Insurance Model LASI	`insurance_model_lasi`	✓
86	Level Coverage LASI	`level_coverage_lasi`	✓
87	Insurance model LIASI	`insurance_model_liasi`	✓
88	Level Coverage LIASI	`level_coverage_liasi`	✓
89	Accident coverage LIASI	`accident_coverage`	✓
90	Insurance model LSISI	`insurance_model_lsisi`	✓
91	Level Coverage LSISI	`level_coverage_lsisi`	✓
94	Permit	`permit`	✓
96	Group Mutuel Template	`template_groupe_mutuel_template`	✓
99	Nur KVG	`template_nur_kvg`	✓
108	test template1	`template_test_template1`	✓
109	test 2	`template_test_2`	✓
110	test5	`template_test5`	✓
111	test 6	`template_test_6`	✓
113	test 7	`template_test_7`	✓
114	test 8	`template_test_8`	✓
116	test 89	`template_test_89`	✓

ID	NAME	HANDLE	TABLE
117	Test52526373	`template_test52526373`	✓
118	test 9	`template_test_9`	✓
119	test 10	`template_test_10`	✓
120	test 11	`template_test_11`	✓
121	Test765343678697867565	`template_test765343678697867565`	✓
122	Group Mutuel Template 2.0	`template_group_mutuel_template_20`	✓

E Source inventory

TrustCore's own source files by size. Framework scaffolding and the ngx-admin demo pages are excluded.

Laravel application, routes and migrations

FILE	LINES
app/Http/Controllers/CollectionController.php	795
app/Http/Controllers/TemplatesController.php	624
app/Http/Controllers/CollectionMetaController.php	543
app/Http/Controllers/UserController.php	266
app/Traits/GetRelationalFieldDetail.php	200
app/Http/Controllers/SettingsController.php	177
app/Http/Controllers/Auth/RegisteredUserController.php	124
routes/api.php	100
app/Http/Controllers/SmsController.php	100
app/Http/Requests/Auth/LoginRequest.php	85
app/Traits/DynamicValidationRules.php	78
app/Http/Kernel.php	72
app/Models/User.php	67
app/Http/Controllers/Auth/AuthenticatedSessionController.php	59
database/migrations/0001_01_01_000002_create_jobs_table.php	57
database/migrations/0001_01_01_000000_create_users_table.php	53
app/Http/Controllers/Auth/NewPasswordController.php	53
app/Http/Controllers/RolesController.php	40
app/Http/Controllers/Auth/PasswordResetLinkController.php	39
routes/auth.php	37
database/migrations/2025_01_10_141633_drop_type_column_from_collections_table.php	36
app/Models/UserMeta.php	36
database/migrations/0001_01_01_000001_create_cache_table.php	35
database/migrations/2025_01_01_071004_create_personal_access_tokens_table.php	33
database/migrations/2025_01_07_065111_create_user_metas_table.php	32
database/migrations/2025_02_25_122241_create_templates_meta_table.php	31
database/migrations/2025_01_06_124755_create_collections_table.php	31
app/Http/Controllers/Auth/VerifyEmailController.php	31
database/migrations/2025_01_01_071533_create_user_roles_table.php	30
database/migrations/2025_02_04_074306_create_settings_table.php	29
database/migrations/2025_01_09_133818_add_last_login_and_settings_to_user_metas_table.php	29
database/migrations/2025_02_25_121855_create_templates_table.php	28
database/migrations/2025_01_01_072018_add_name_to_roles_table.php	28
database/migrations/2025_01_01_071412_create_roles_table.php	28

FILE	LINES
app/Providers/AppServiceProvider.php	27
app/Http/Middleware/EnsureEmailIsVerified.php	27
app/Models/TemplatesMeta.php	26
app/Models/Collections.php	25
app/Http/Controllers/Auth/EmailVerificationNotificationController.php	25
app/Models/Templates.php	22
app/Models/Role.php	21
app/Models/Setting.php	10
routes/web.php	9
routes/console.php	8
app/Http/Controllers/Controller.php	8

Angular — TrustCore code

FILE	LINES
src/app/components/users/users.component.ts	1,012
src/app/components/collections/collection-table/collection-table.component.ts	520
src/app/components/entryedit/entry.component.ts	489
src/app/components/blueprint/blueprint.component.ts	372
src/app/components/collections/collection-listing/collection-listing.component.ts	281
src/app/pages/pages-routing.module.ts	269
src/app/services/users.service.ts	216
src/app/components/sitesettings/sitesettings.component.ts	196
src/app/components/usersettings/usersettings.component.ts	179
src/app/pages/pages-menu.ts	171
src/app/pages/users/create-user/create-user.component.ts	166
src/app/components/roles/roles.component.ts	130
src/app/auth/custom-password-auth-strategy.ts	115
src/app/services/collections.service.ts	99
src/app/auth.guard.ts	82
src/app/components/collections/collection-table/collection-table.component-bk.ts	73
src/app/pages/users/edit-user/edit-user.component.ts	72
src/app/components/datepicker/datepicker.component.ts	59
src/app/components/collections/tables-routing.module.ts	51
src/app/components/link-render/link-render.component.ts	47
src/app/services/blueprint.service.ts	43
src/app/services/site-settings.service.ts	37
src/app/components/collections/tables.module.ts	34
src/app/components/collections/collection-listing/custom-link-render.component.ts	29
src/app/components/fileupload/file-upload-editor.component.ts	22
src/app/components/entryedit/entry.module.ts	22

FILE	LINES
src/app/pages/users/create-user/create-user.module.ts	21
src/app/pages/users/edit-user/edit-user.module.ts	19
src/app/shared/shared.module.ts	16
src/app/services/main.service.ts	15
src/app/pages/users/users.module.ts	15
src/app/pages/users/edit-user/edit-user-routing.module.ts	15
src/app/pages/users/create-user/create-user-routing.module.ts	15
src/app/pipes/filter.pipe.ts	12
src/app/components/collections/tables.component.ts	8

F Glossary

Terms used throughout this manual.

TERM	MEANING IN TRUSTCORE
Blueprint	The field builder screen for a collection; also the source template chosen when creating a new template.
Collection	A registered content type with its own physical table `{handle}_meta`.
Column grammar	The naming convention that encodes type, required flag and relation target in a column name.
Company Credentials	The `insurance_companies` collection holding advisors' insurer portal logins.
Desktop flow / PAD	Power Automate Desktop robot that drives insurer portals unattended.
Cloud flow	Power Automate (Logic Apps) workflow triggered over HTTP with an offer id.
Entry	One row of a collection's table.
Handle	Slugged, immutable identifier of a collection; also part of its table name.
KVG / LAMal	Swiss mandatory basic health insurance (Krankenversicherungsgesetz / Loi sur l'assurance-maladie).
VVG / LCA	Swiss supplemental insurance governed by the Insurance Contract Act.
Lookup	A small collection used as a dropdown source (models, levels, deductibles…).
Offer	A customer-facing insurance proposal: customer + template, completed with a PDF.
OTP	One-time password received by SMS during insurer portal sign-in.
Product switch	A `rel_…_col_boolean` field that includes or excludes a product line in a template.
Sanctum	Laravel package issuing personal access tokens used as bearer tokens.
Scoping	Restricting rows by owner or assigned advisor.
Template	A collection with prefix `template_` whose single row configures the product lines of offers.